WebBeans User's Guide for Developers

2026-10-28

Contents

0. Before you start

1. Your first window

2. Components, values and events

2.1 Value components and validation
2.2 Two kinds of text events
2.3 ComboBox
2.4 Numbers, dates, choices — and a Form
2.5 The rest of the basics
2.6 Enums look pretty by default

3. Layouts

3.1 SplitPane and ScrollPane
3.2 Files in and out

4. The session thread, timers and background work

5. Talking to the user: status, notifications, dialogs

5.1 Dialogs are asynchronous
5.2 Your own dialog content and buttons
5.3 Wizards

6. Windows in depth

6.1 Lifecycle and state
6.2 Where it docks: modes and your own
6.3 Many instances of one window: editors
6.4 The window’s lookup: how the rest of the platform sees your window

6a. Tables of plain objects

6a.1 Trees of plain objects: TreeTable
6a.2 Editing one object: PropertyGrid

6b. Text, code, boards and viewers

7. Nodes and explorer views

7.1 A node per object
7.2 Children
7.3 The explorer window

8. Actions, menus, toolbars, shortcuts and popups

9. Options and preferences

10. Charts

11. 3D scenes

11a. Graphs and maps

12. Services, sessions and server push

12.1 Three scopes
12.2 The session bus: publish/subscribe across windows and tabs
12.3 Push from a background thread
12.4 Who is logged in

13. Internationalisation

14. Testing without a browser

15. Help pages

15.1 Connecting pages to the UI
15.2 Writing pages
15.3 Outside the window

16. Packaging an application

17. The patterns, on one page

*A step-by-step introduction to the UI components, the platform patterns and the way a WebBeans application is put together. Every example compiles and runs: the small ones are the `webbeans-tutorial` jar (`webbeans-tutorial/src/main/java/org/webbeans/tutorial`, built and tested by `./build.sh` and on the class path next to the demo), the larger ones are taken verbatim from the **Workbench** sample application (`workbench/`) and the demo (`webbeans-demo/`). All code is Java 8.*

Companion documents: the **Developer Guide** (`docs/developer-guide.md`) explains the concepts in depth; the **Component Guide** (`docs/component-guide.md`) is the per-component reference. This guide is the path through them.

The screenshots in this guide were taken from the demo application (`webbeans-demo` + `webbeans-tutorial`, started as in §0) at 1280×800 in a stock browser theme; the Workbench looks the same with different windows. They are in `docs/images/`.

0. Before you start

What WebBeans is. A rich-client platform in the spirit of the NetBeans Platform, where *everything* is server-side Java: windows, components, menus, dialogs, nodes, charts and 3D scenes are Java objects living in a per-user session on the server. A small generated JavaScript client renders them in the browser and sends events back. You never write JavaScript; you write ordinary Java jars, and whatever is on the class path is the application.

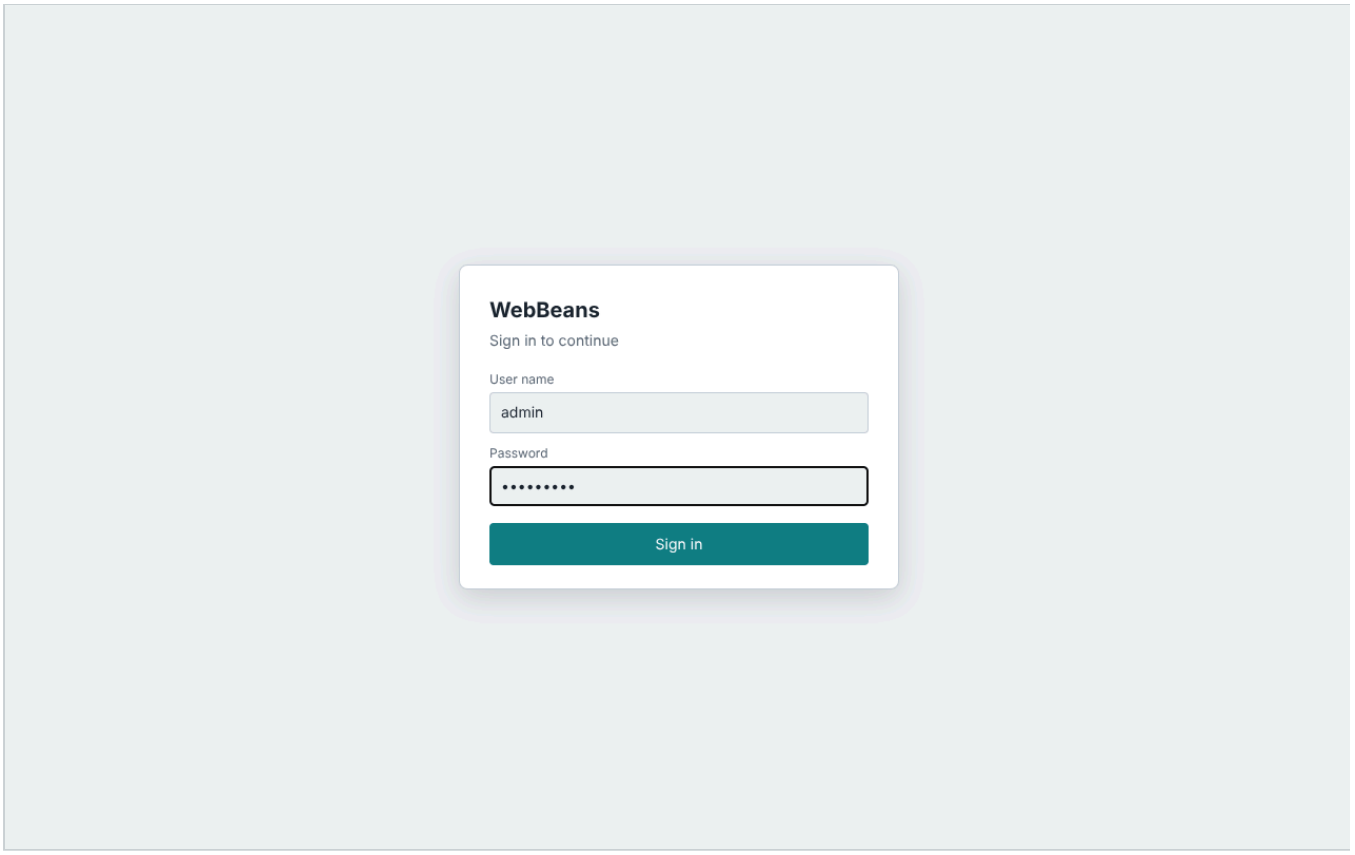
The three ideas you will use on every page of this guide:

1. **Components are Java objects with properties and events.** `new Button("Save")`, `button.setEnabled(false)`, `button.addClickListener(...)`. A change on the server becomes a *delta* to the browser; a click in the browser becomes an event on the server.
2. **Everything is registered by annotation.** `@TopComponent.Registration`, `@ActionRegistration`, `@OptionsPanelController.TopLevelRegistration`, `@HelpPage`, `@ServiceProvider`… An annotation processor writes the entries into a *registry* (folders such as `Menu/File`, `Windows/Modes/editor`, `Toolbars/File`, `Help/Pages`) at compile time. There is no `layer.xml`.
3. **One thread per session.** Your code touches components only on the session's executor — inside an event listener, a timer, or `ui.access(Runnable)`. Background work happens elsewhere and hops back with `access`.

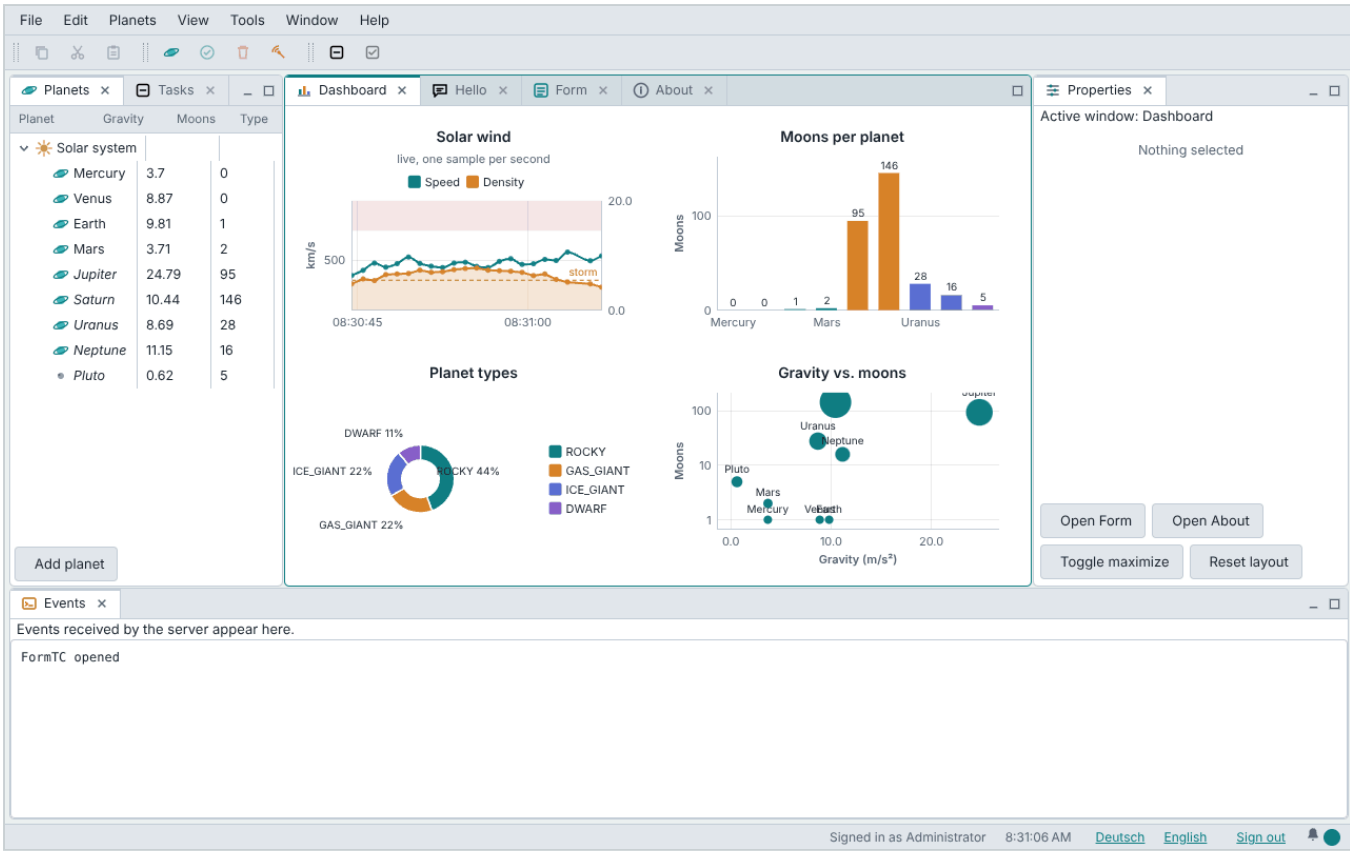
Build and run. From the repository root:

```
./build.sh                       # compiles every project for Java 8, runs the JUnit tests
# the demo + the tutorial (both jars are in build/dist/lib with the platform):
java -cp 'build/dist/lib/*' org.webbeans.launcher.Main --datadir data --port 8080
# the Workbench application (one merged jar):
cd build/workbench-dist && ./workbench.sh            # http://localhost:8080, alice/secret (admin) or
bob/secret
```

The demo's first start prints a generated `admin` password (or pass `--admin-password …`). Log in and you will see the main window: menu bar, toolbars, the dock with the open windows, the status bar. Everything you see was produced by annotations in the jars on the class path.



The login page — the platform's default `LoginPage`; register your own implementation of `org.webbeans.auth.spi.LoginPage` for a branded one (see `design/authentication.md`).



The demo's main window. Left: the explorer *mode with the Planets and Tasks windows; centre: the* editor *mode with the Dashboard, Hello, Form and About tabs; right: the* properties *mode; bottom: the* output *mode with the Events window. Menu bar and toolbars above, the status bar with the signed-in user, the clock, the language switch and the connection indicator below.*

Anatomy of an application jar. Nothing special: an ordinary jar whose classes carry the WebBeans annotations. The annotation processor turns them into `META-INF/services` entries at compile time, and the platform reads those from every jar on the class path at start-up — there is no manifest, no descriptor and no module system.

```
webbeans-tutorial/
  src/main/java/org/webbeans/tutorial/*.java       # @Messages on these generate Bundle-
generated.properties
  src/main/java/org/webbeans/tutorial/package-info.java   # package-level registrations (help pages)
  src/main/resources/org/webbeans/tutorial/*.svg    # icons
  src/main/resources/org/webbeans/tutorial/help/*.md      # help pages (§15)
  src/test/java/org/webbeans/tutorial/TutorialTest.java
```

Compile with the three annotation processors on the processor path and `-Awebbeans.module=<name>` (the package of the generated registrations class; see `build.sh`, `examples/hello/pom.xml` or any project `pom.xml`); put the jar on the class path and start. An `Installer` subclass registered with `@ServiceProvider(service = Installer.class)` gets `restored()` at start-up and `closing()`/`close()` at shutdown (the Workbench’s `WorkbenchInstaller` seeds the demo accounts). Jars depend on each other the ordinary Maven way; the smallest complete application is `examples/hello` (a pom and one action class).

1. Your first window

A **TopComponent** is a window in the dock: it has a display name, an icon, a mode (where it docks) and content. Here is the whole of `HelloTopComponent`:

```
@TopComponent.Registration(mode = "editor", openAtStartup = true, position = 50)
@TopComponent.Description(preferredID = "HelloTC", iconBase = "org/webbeans/tutorial/hello.svg",
helpId = "tutorial")
@TopComponent.OpenActionRegistration(displayName = "#CTL_OpenHello", position = 50)
@Messages({"CTL_HelloTC=Hello", "CTL_OpenHello=Hello", "LBL_YourName=Your name", "BTN_Greet=Greet",
"MSG_Hello=Hello, {0}!"})
public final class HelloTopComponent extends TopComponent {

    private final TextField name = new TextField();
    private final Label greeting = new Label();

    public HelloTopComponent() {
        setDisplayName(Bundle.CTL_HelloTC());
        setLayout(new Layout.Flow(Layout.Orientation.VERTICAL, 8));
        setPadding(12);
        name.setPlaceholder(Bundle.LBL_YourName()).setTestId("hello-name");
        Button greet = new Button(Bundle.BTN_Greet());
        greet.setKind(Button.Kind.PRIMARY).setDefaultButton(true).setTestId("hello-greet");
        greet.addClickListener(new Listener<ClickEvent>() {
            @Override
            public void onEvent(ClickEvent e) {
                greeting.setText(Bundle.MSG_Hello(name.getValue()));
            }
        });
        greeting.setTestId("hello-greeting");
        add(name);
        add(greet);
        add(greeting);
    }

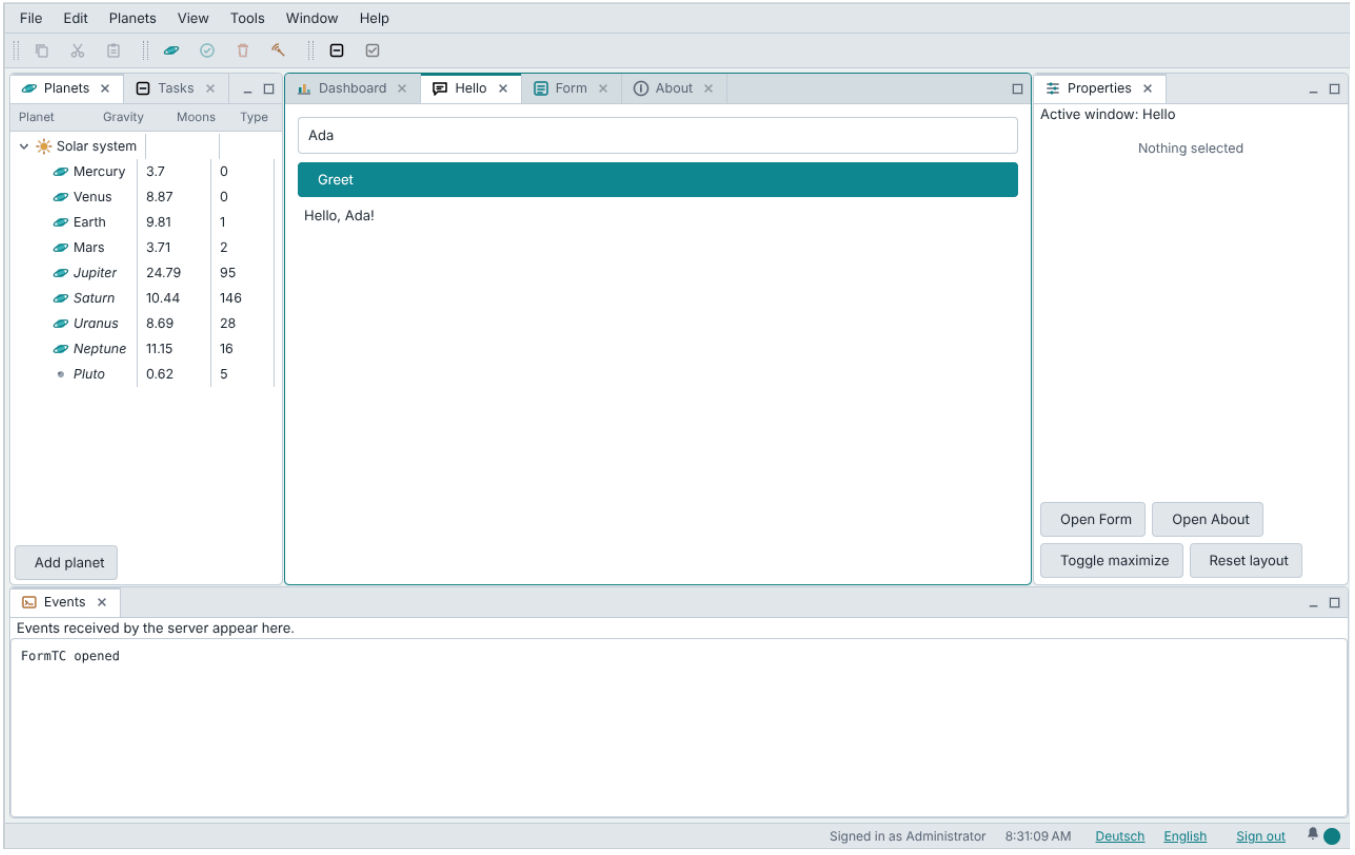
    @Override
    protected void localeChanged() {
        setDisplayName(Bundle.CTL_HelloTC());
    }
}
```

What each line buys you:

- `@TopComponent.Registration(mode = "editor", openAtStartup = true)` — docks the window in the central *editor* mode and opens it when the user logs in. The standard modes are `explorer` (left), `navigator` (left), `editor` (center), `properties` (right), `output` (bottom); §6 shows how to add your own.
- `@TopComponent.Description(preferredID, iconBase, helpId)` — the stable id (used by `WindowManager.findTopComponent`, saved layouts and tests), the tab icon, and the help page shown when the user presses F1 in this window (§15).
- `@TopComponent.OpenActionRegistration(displayName)` — an entry in the **Window** menu that opens (or re-activates) the window.
- `@Messages({...})` — generates `Bundle.properties` and a `Bundle` class with one static method per key (`Bundle.MSG_Hello("Ada")` formats `{0}`). `#CTL_OpenHello` in an annotation refers to a key. Put `Bundle_de.properties` next to the class for German, and the platform picks the user's locale (§13).

- `setTestId("hello-name")` — a stable `data-test` attribute; §14 drives the window through it from a JUnit test, and the Playwright scripts use the same ids.
- The constructor is the whole UI. There is no render phase: components attached to an open window are sent to the browser as they change.
- `localeChanged()` — called when the user switches language; relabel what you set in the constructor.

Build, restart, log in: the window is there, with a *Window ▸ Hello* entry. Type a name, press Enter (the default button) and the label updates — one event up, one delta down.

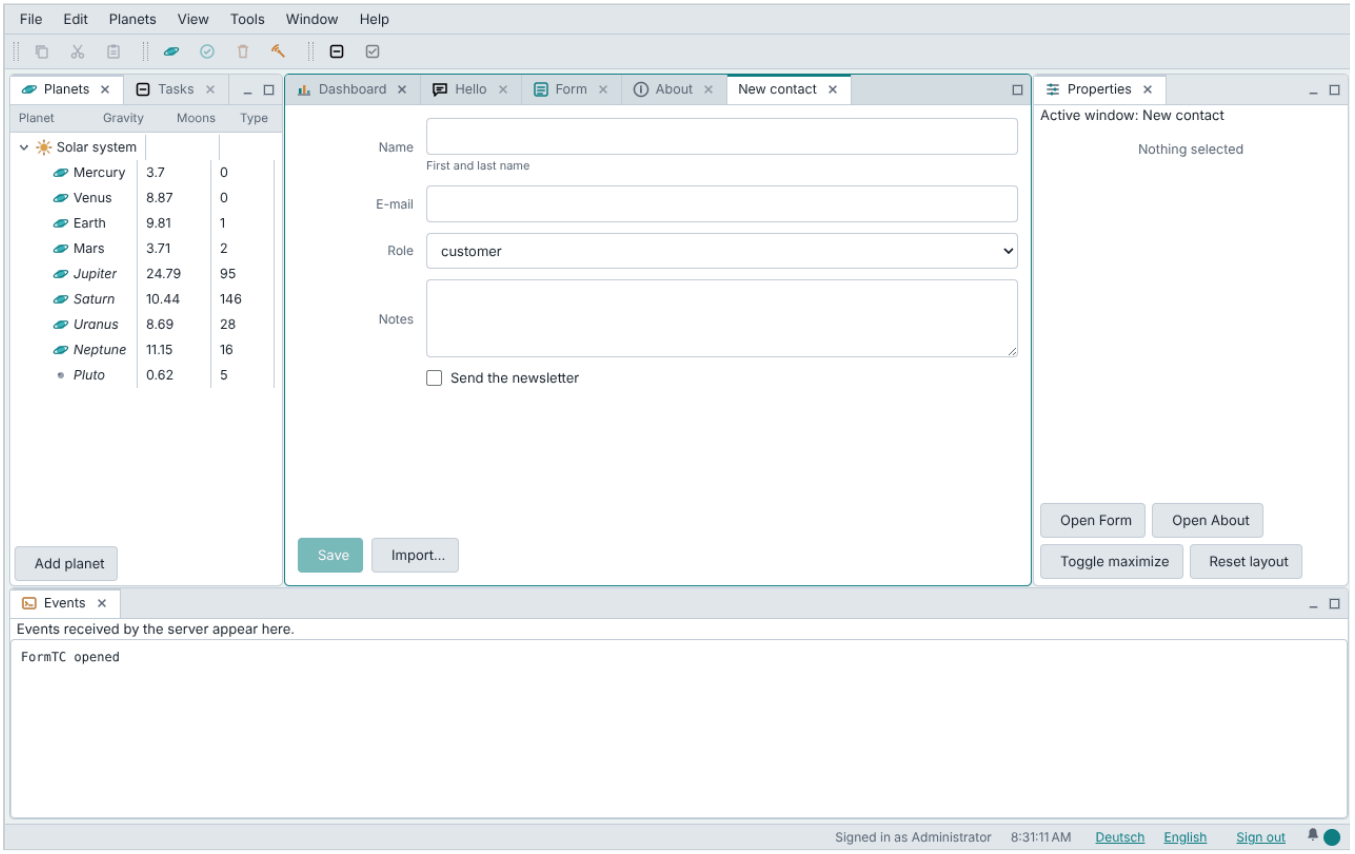


`HelloTopComponent` in the editor mode: a vertical flow of a text field, a primary button and a label — the whole constructor above, rendered.

Pattern — read values when you need them, not on every keystroke. `name.getValue()` is up to date because the text field's value is *client-owned*: the browser reports it on commit (blur/Enter) and the server keeps a copy. The server never pushes the value back while the user types, so there are no cursor jumps.

2. Components, values and events

The tutorial's `ContactTopComponent` uses most of the input components at once (open it with *Window ▸ New contact*):



`ContactTopComponent`: a `Form` with captions, the "First and last name" help text under the first field, a `ComboBox`, a `TextArea`, a `CheckBox` and the button row. Save is disabled until the form validates.

```
private final TextField name = new TextField();
private final TextField email = new TextField();
private final ComboBox<String> role = new ComboBox<String>();
private final TextArea notes = new TextArea();
private final CheckBox newsletter = new CheckBox(Bundle.LBL_Newsletter());
private final Button save = new Button(Bundle.BTN_Save());
private final ProgressBar progress = new ProgressBar();
```

2.1 Value components and validation

`TextField`, `TextArea`, `NumberField`, `CheckBox`, `ComboBox<T>`, `RadioGroup<T>`, `Slider`, `DatePicker` and `DateTimePicker` are **value components** (`ValueComponent<T>`): `getValue()`, `setValue(v)`, `addChangeListener(...)`, `setRequired(true)` (the platform shows "Required" itself), `setReadOnly(true)`, `validate()`/`isValid()` and a `Validator<T>` that returns an error message or `null`:

```
name.setRequired(true).setValidator(new Validator<String>() {
    @Override
    public String validate(String v) {
        return v == null || v.trim().isEmpty() ? Bundle.ERR_NameRequired() : null;
    }
});
email.setValidator(new Validator<String>() {
    @Override
    public String validate(String v) {
        return v == null || v.isEmpty() || v.matches("[^@\\s]+@[^@\\s]+") ? null :
Bundle.ERR_Email();
    }
});
```

The message appears under the field in the browser; the change event tells you whether the value passed:

```
save.setKind(Button.Kind.PRIMARY).setDefaultButton(true).setEnabled(false);
name.addChangeListener(new Listener<ChangeEvent<String>>() {
    @Override
    public void onEvent(ChangeEvent<String> e) {
        save.setEnabled(e.isValid());          // e.getValue(), e.getOldValue() are there too
    }
});
```

Pattern — enable the action, don’t check in the action. Bind the Save button’s enabled state to validity, so the user sees what is missing before clicking rather than after.

2.2 Two kinds of text events

`addChangeListener` fires on commit. If you need every keystroke (live search, “n characters left”), use the debounced **input** event:

```
name.addInputListener(new Listener<TextField.InputEvent>() {
    @Override
    public void onEvent(TextField.InputEvent e) { validate(e.getValue()); }   // 300 ms after the
last key
});
```

The demo’s `PlanetWizardAction.NamePanel` uses both: input to update the wizard’s error line while typing, change to validate on commit. The Help window’s search field does the same, so the results follow the typing.

2.3 ComboBox

A `ComboBox<T>` holds your objects, not strings. Give it a label function (and optionally an icon function); keys default to `toString()`:

```
role.setItems(Arrays.asList("customer", "supplier",
"partner")).setAllowEmpty(false).setValue("customer");

planet.setItems(SolarSystem.get().getPlanets(), new Function<Planet, String>() {     // demo
FormTopComponent
    @Override public String apply(Planet p) { return p.getName(); }
});
planet.setValue(SolarSystem.get().find("Earth"));
planet.setFilterable(true);                       // type to filter long lists
```

2.4 Numbers, dates, choices — and a Form

The demo's *Components* window (`GalleryTopComponent`, *Window* ▸ *Components*) puts the remaining inputs on one form:

```
NumberField quantity = new NumberField(Double.valueOf(1));
quantity.setIntegerOnly().setRange(1, 999).setSpinner(true).setRequired(true);
NumberField price = new NumberField(Double.valueOf(19.99));
price.setDecimals(2).setPrefix("€").setMin(Double.valueOf(0));          // 1,234.5 for an English
user, 1.234,5 for a German one
Slider discount = new Slider(0, 50, 10);
discount.setStep(5).setTicks(Double.valueOf(10)).setShowValue(true);
RadioGroup<String> shipping = new RadioGroup<String>(Arrays.asList("standard", "express",
"collect"));
shipping.setItems(Arrays.asList("standard", "express", "collect"),
labelFn).setValue("standard").setOrientation(RadioGroup.Orientation.HORIZONTAL);
DatePicker delivery = new DatePicker(LocalDate.now().plusDays(3));
delivery.setMin(LocalDate.now()).setClearButton(true);
DateTimePicker pickup = new DateTimePicker(LocalDateTime.now().plusHours(2));
```

Numbers are `Double`s (`getIntValue()` rounds), dates are `java.time` values, choices are your objects — the browser only ever sees formatted text and keys. Ranges are validated on the server with localised messages ("Must be at most 999"); text that is not a number or a date is rejected and the last valid value comes back.

A **`Form`** collects fields and keeps a button in step with their validity, which is what §2.1 did by hand for one field:

```
final Button place = new Button(Bundle.BTN_Place());
final Form form = new Form().add(quantity, price, discount, shipping, delivery,
pickup).enable(place);
place.addClickListener(new Listener<ClickEvent>() {
    @Override public void onEvent(ClickEvent e) {
        if (form.validate()) {                                  // also checks the fields the user
never touched
            summary.setText(Bundle.MSG_Order(quantity.getIntValue(), price.format(price.getValue()),
...));
        }
    }
});
```

`form.bind(field, consumer)` plus `form.write()` moves the values into your model; `form.reset()` clears the errors after a save. A required field that is still empty keeps the button disabled without showing an error until the user touches it.

Pattern — let the server own the rules. Min/max, required, formats: declare them on the component and the platform validates, localises and re-sends; your listener only sees `isValid()`.

2.5 The rest of the basics

Component	One-liner
`Label`	`new Label(text)`, `setHtml("<b>3</b> warnings")`, `setIcon(Resource.of("…/ok.svg"))`
`Html`	rich read-only text (`AboutAction` shows one in a dialog)
`Link`	`new Link(text, href)`; without href it fires `CLICK`
`Image`	`new Image(Resource.of("…/logo.svg"), "alt")`
`ProgressBar`	`setValue(0.4)`, `setText("40 %")`, `setIndeterminate(true)`
`Spacer`	pushes the following components of a `Layout.Flow` row to the end
`Upload`	a file button with a drop zone; files arrive as `UploadEvent`s on the session thread (`e.getFile().open()`), see §3.2
`ListBox<T>`	a list of your objects with single/multi selection; `getSelected()`
`Tree<T>`	a lazily loaded tree over a `TreeModel<T>` (children fetched on first expansion); for nodes with actions use `TreeView` (§7)
`ToggleButton`, `ButtonGroup<T>`	a button that stays pressed (a `Boolean` value); a segmented control over your objects or an enum (`ButtonGroup.of(Type.class)`)
`SplitButton`	a main action plus a small menu of alternatives (`addItem(key, label)` → `addSelectListener`)
`ColorPicker`, `TimePicker`	a `#rrggbb` string from a palette or the browser's colour dialog; a `LocalTime` with `setMin`/`setMax`
`PasswordField`	a masked `TextField` with a reveal button and a strength meter (`PasswordField.strength(text)` for your validator)
`TagInput`	a `List<String>` entered as chips, with suggestions and `setMaxTags`
`Badge`, `Chip`, `Avatar`	a counter or status pill; a closable/selectable token; initials or a picture with a presence dot
`EmptyState`, `Breadcrumb`, `Stepper`	the placeholder of an empty view (with an action); a clickable path; numbered steps with a current one
`Card`, `Accordion`, `Popover`	a panel drawn as a card (header, optional banner, clickable); collapsible sections; a floating panel anchored to another component
`panel.setBusy(true, "Scanning…")`	dims any panel under a spinner while a long operation runs (§4)

All of these are in the demo's *Widgets* window (`WidgetsTopComponent`), each wired to a label that shows the last event.

Every component has `setVisible`, `setEnabled`, `setTooltip`, `setStyleName("my-css-class")`, `setWidth(Size.px(200))`/`setHeight`, `setTestId`, and generic `get(Property)`/`set(Property, value)` for the rare property without a setter (`text.set(TextArea.WRAP, Boolean.FALSE)`).

Icons are resources in your jar: `iconBase = "org/webbeans/tutorial/task.svg"` in annotations, `Resource.of(path)` in code. Prefer SVG; it follows the theme.

2.6 Enums look pretty by default

Wherever an enum value is shown — `ComboBox`, `RadioGroup`, `ButtonGroup`, `ListBox` items, `Table` cells, a `PropertyGrid` or a node property in the Properties window and `OutlineView` — the platform asks `EnumLabels.label(value)` (and `EnumLabels.icon(value)`) instead of calling `toString()`. The lookup order:

1. the enum implements `org.webbeans.ui.Labeled` (`getDisplayName()`, optional `getIcon()` and `getDescription()`) — the demo's `Planet.Type` returns `@Messages` texts and an icon per constant;
2. a function registered once with `EnumLabels.register(Type.class, labelFn[, iconFn])` — for enums from a library;
3. the key `<SimpleName>.<CONSTANT>` in the `Bundle.properties` of the enum's package (`Type.GAS_GIANT=Gas giant`, with `Bundle_de.properties` for German);
4. failing all that, the constant name prettified: `GAS_GIANT` → *Gas giant*.

So a plain enum already reads well, a bundle line localises it, and `Labeled` or `register` add icons. Typed input (`EnumLabels.parse`, the editors' `fromText`) accepts the name or the label. Node properties can go further: `PropertyEditors.registerEnum(Type.class, label, icon)` replaces the editor for a type, and `property.setRenderer(Function<T, Cell>)` draws one property's value as a styled `Cell` (icon, html, colour, tooltip) while the text stays editable — §7.1.

3. Layouts

A `Panel` (and a `TopComponent`) arranges its children with a `Layout`. You choose the layout with `setLayout` / the constructor and pass a *constraint* when adding:

```
// a two-column form: label/field rows with an optional hint
Panel form = new Panel(new Layout.Form().labelWidth(Size.px(110)));
form.add(name, Layout.Form.row(Bundle.LBL_Name(), Bundle.HINT_Name()));
form.add(email, Layout.Form.row(Bundle.LBL_Email()));
form.add(role, Layout.Form.row(Bundle.LBL_Role()));
form.add(notes, Layout.Form.row(Bundle.LBL_Notes()));
form.add(newsletter, Layout.Form.row(""));          // or form.addRow(label, field)

// a button row: Spacer pushes the progress bar to the right
Panel buttons = new Panel(new Layout.Flow(Layout.Orientation.HORIZONTAL, 8));
buttons.add(save);
buttons.add(importButton);
buttons.add(new Spacer());
buttons.add(progress);

// the window: form in the middle, buttons at the bottom
setLayout(new Layout.Border());
setPadding(12);
add(form, Layout.Border.CENTER);
add(buttons, Layout.Border.SOUTH);
```

Layout	Use it for	Constraint
`Layout.Border`	a window: toolbar on top, content in the middle, status at the bottom	`Layout.Border.NORTH/SOUTH/EAST/WEST/CENTER`
`Layout.Flow(orientation, gap)`	button rows, vertical stacks; `.wrap(true)`, `.align(...)`	none (`Spacer` for the gap)
`Layout.Form`	label/field forms, `.labelWidth(...)`, `Layout.Form.section(title)`	`Layout.Form.row(label[, hint])`
`Layout.Grid(rows, cols, hgap, vgap)`	dashboards; `.fill()` gives every cell the same share	none (row-major) or `Layout.Grid.cell(row, col, rowSpan, colSpan)`
`Layout.Stack`	one child visible at a time (`panel.setSelected(child)`) — wizards, option pages, the Help window's contents/page switch	none
`Layout.Fill`	one child fills the panel	none

Tabs: `TabbedPane tabs = new TabbedPane(); tabs.addTab(title, content); tabs.addTab(title, icon, content, closable)`. Scrolling: `setScrollable(true)` on the panel or window. The Workbench dashboard is a `Layout.Grid(2, 2, 8, 8).fill()` with three charts added row-major — four lines of layout code for a responsive dashboard.

Pattern — compose panels, don't nest layouts in your head. Build the form panel, the button panel and the content panel separately (each a field or a local), then place them with one `Layout.Border`. It reads top-down and each piece can be moved or reused.

3.1 SplitPane and ScrollPane

```
SplitPane split = new SplitPane(SplitPane.Orientation.HORIZONTAL, tree, details);   // first child =
left/top pane
split.setPosition(240).setCollapsible(true);                                          // the user
drags it; POSITION is client-owned
ScrollPane scroll = new ScrollPane(cities);                                          // reports
SCROLL_X/SCROLL_Y, scrollTo(), scrollToBottom()
```

`SplitPane` is for splits *inside* a window (the window system draws the splitters between modes itself); `ScrollPane` when you need the scroll offsets or `scrollToBottom()` — otherwise `setScrollable(true)` on a `Panel` is enough.

3.2 Files in and out

```
Upload up = new Upload();
up.setAccept(".csv,text/csv").setMaxSize(5L * 1024 * 1024);
up.addUploadListener(new Listener<Upload.UploadEvent>() {
    @Override public void onEvent(Upload.UploadEvent e) {
        importCsv(e.getFile().open());                      // a temp file, deleted when this
returns unless e.getFile().retain()
    }
});
DownloadService.offer(csvBytes, "orders.csv", "text/csv");  // the browser saves a file, once
panel.add(DownloadService.link(producer, "Invoice (PDF)"));    // a Link that produces the file on
every click
```

The browser posts the file straight to the server (`/wb/upload`); the event arrives on the session executor like any other, so you can touch components in it. Downloads are streamed by the server from a one-time URL — nothing large goes through the UI channel.

4. The session thread, timers and background work

Every session has one executor; all your UI code runs on it. Listeners, dialog callbacks and timers are already there. Two cases need care:

Periodic work on the UI — use the UI's timer; it runs on the session thread and may touch components directly:

```
private void startImport() {                                    // ContactTopComponent
    progress.setVisible(true);
    progress.setValue(0).setText(Bundle.MSG_Importing(0));
    makeBusy(true);                                             // a spinner on the window tab
    final int[] pct = {0};
    final UI.Timer[] timer = new UI.Timer[1];
    timer[0] = getUI().scheduleAtFixedRate(new Runnable() {
        @Override
        public void run() {
            pct[0] += 25;
            progress.setValue(pct[0] / 100.0).setText(Bundle.MSG_Importing(pct[0]));
            if (pct[0] >= 100) {
                timer[0].cancel();
                makeBusy(false);
                progress.setText(Bundle.MSG_Imported());
            }
        }
    }, 200, 200);
}
```

The Workbench dashboard samples its metrics the same way (`getUI().scheduleAtFixedRate(..., 0, seconds * 1000L)`) and cancels the timer in `componentClosed()`. Always cancel timers you started; a closed window with a live timer keeps pushing deltas nobody sees.

Long work off the UI — do it on your own thread (or an action marked `asynchronous = true`, §8) and hop back with `access`:

```
final UI ui = e.getUI();                                        // demo ScanAction
final UISession session = ui.getSession();
ProgressHandle h = ProgressHandleFactory.createHandle(Bundle.CTL_Scan(), new Cancellable() {
    @Override public boolean cancel() { cancelled[0] = true; return true; }
}, session);
h.start(sectors);
for (int i = 0; i < sectors && !cancelled[0]; i++) {
    h.progress(Bundle.MSG_Scanning(i + 1), i);                  // thread-safe: shows in the status
bar of every tab
    Thread.sleep(stepMillis);
}
h.finish();
ui.access(new Runnable() {                                      // back on the session thread
    @Override public void run() { session.publish(new LogLine(msg)); }
});
```

`ui.access` and `session.access` return a `Future` you can wait on from a test. Strict threading is on by default (`UISession.setStrictThreading`): touching a component from the wrong thread throws at the call site instead of corrupting the delta stream, so mistakes show up in the first test run.

5. Talking to the user: status, notifications, dialogs

Three channels, in increasing weight:

```
StatusDisplayer.getDefault().setStatusText(Bundle.MSG_Saved(who));                    //
status bar, transient
NotificationDisplayer.getDefault().notify(Bundle.TITLE_Save(), Bundle.MSG_Saved(who),
NotificationDisplayer.Category.INFO);   // a toast + the bell
DialogDisplayer.getDefault().notify(new NotifyDescriptor.Message(html,
NotifyDescriptor.INFORMATION_MESSAGE));   // modal
```

5.1 Dialogs are asynchronous

There is no blocking `showDialog()`; the browser answers later. `notify` returns a `CompletableFuture<Object>` resolved with the chosen option, on the session thread:

```
private void confirmAndSave() {                                                       //
ContactTopComponent
    final String who = name.getValue().trim();
    NotifyDescriptor.Confirmation ask = new NotifyDescriptor.Confirmation(Bundle.MSG_Confirm(who,
role.getValue()),
            Bundle.TITLE_Save(), NotifyDescriptor.YES_NO_OPTION);
    DialogDisplayer.getDefault().notify(ask).thenAccept(new Consumer<Object>() {
        @Override
        public void accept(Object answer) {
            if (answer == NotifyDescriptor.YES_OPTION) {
                StatusDisplayer.getDefault().setStatusText(Bundle.MSG_Saved(who));
                NotificationDisplayer.getDefault().notify(Bundle.TITLE_Save(), Bundle.MSG_Saved(who),
NotificationDisplayer.Category.INFO);
                setModified(false);
            }
        }
    });
}
```

The ready-made descriptors: `NotifyDescriptor.Message(text-or-component, type)`, `Confirmation(text, title, YES_NO_OPTION | YES_NO_CANCEL_OPTION | OK_CANCEL_OPTION)`, `InputLine(label, title)` (`getInputText()` after OK), `Exception(throwable)`. Answers compare by identity with `NotifyDescriptor.OK_OPTION`, `YES_OPTION`, `NO_OPTION`, `CANCEL_OPTION`, `CLOSED_OPTION`. Any descriptor can carry a help page — `about.setHelpCtx(new HelpCtx("workbench"))` adds a *Help* button at the left of the button bar (the Workbench's About dialog does this; §15).

Pattern — the "save changes?" question on close. `canClose()` cannot wait for an answer, so it asks and returns `false`, and the callback closes the window once the state allows it (`EditorTopComponent`):

```
@Override
protected boolean canClose() {
    if (!dirty) {
        return true;
    }
    DialogDisplayer.getDefault().notify(new
NotifyDescriptor.Confirmation(Bundle.MSG_SaveChanges(path.getFileName().toString()),
            Bundle.TITLE_Close(), NotifyDescriptor.YES_NO_CANCEL_OPTION)).thenAccept(new
Consumer<Object>() {
        @Override
        public void accept(Object r) {
            if (r == NotifyDescriptor.YES_OPTION) {
                try {
                    doSave();
                } catch (IOException e) {
                    DialogDisplayer.getDefault().notify(new NotifyDescriptor.Exception(e));
                    return;
                }
                close();                        // canClose passes now: nothing is modified
            } else if (r == NotifyDescriptor.NO_OPTION) {
                setDirty(false);
                close();
            }
        }
    });
    return false;
}
```

5.2 Your own dialog content and buttons

`DialogDescriptor` takes any component and any options (predefined options, strings — the string is the result — or `Button`s):

```
Panel content = new Panel(new Layout.Flow(Layout.Orientation.VERTICAL, 8));                    // demo
AboutAction
content.add(new Html("<h3 style=\"margin:0\">WebBeans demo</h3><p>A NetBeans-style rich client
platform …</p>"));
DialogDescriptor d = new DialogDescriptor(content, Bundle.TITLE_About(), true, new Object[]
{Bundle.CTL_Close()}, Bundle.CTL_Close(),
        DialogDescriptor.BOTTOM_ALIGN, null);
d.setMessageType(NotifyDescriptor.PLAIN_MESSAGE);
DialogDisplayer.getDefault().notify(d);
```

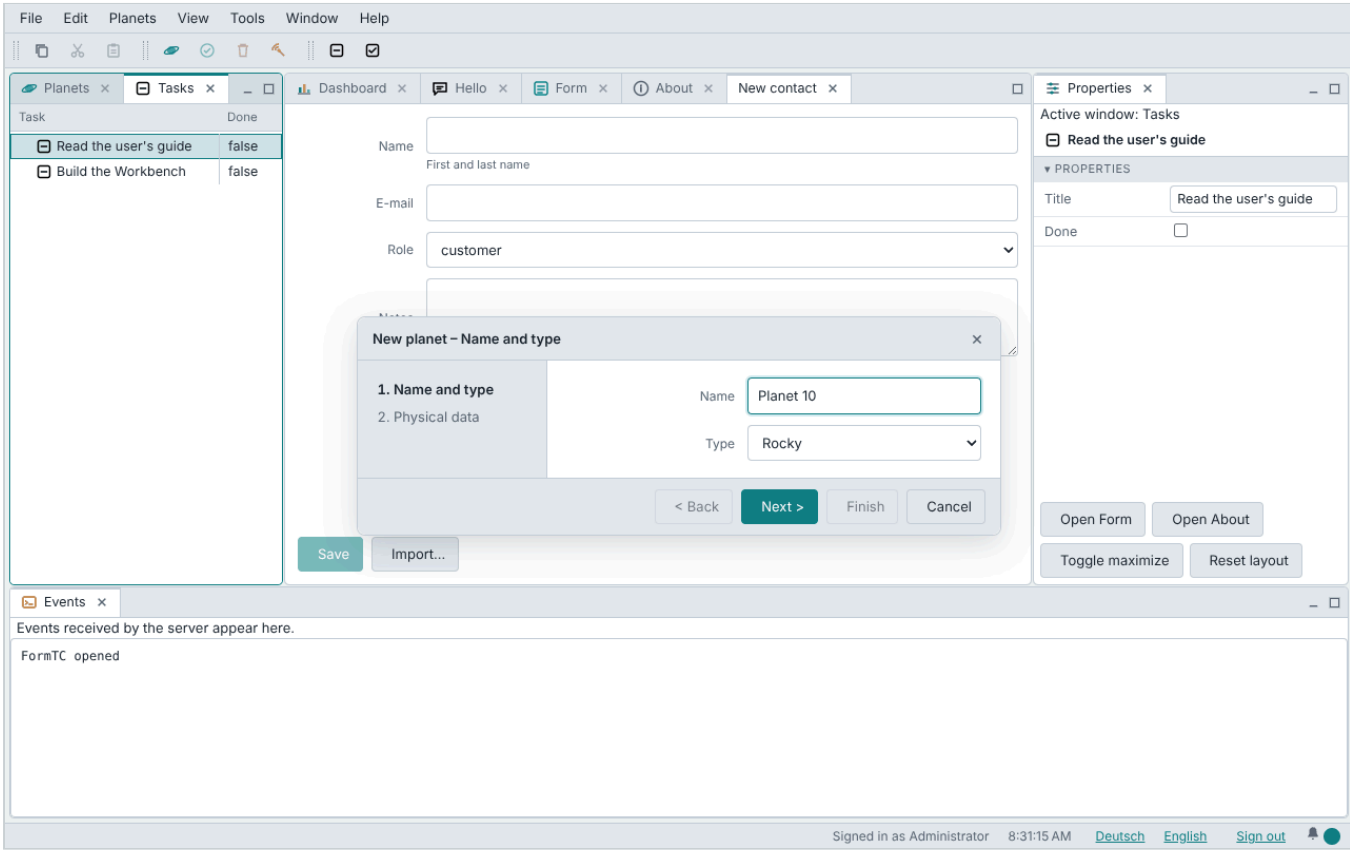
For a form dialog, `d.setValid(false)` disables the default option until your validators pass, `d.createNotificationLineSupport().setErrorMessage(...)` shows a message line, and `setClosingOptions(...)` keeps the dialog open for buttons such as *Apply* (the Options dialog in `DefaultOptionsDisplayer` is a complete example). The NetBeans-shaped constructor with a `HelpCtx` argument exists as well.

5.3 Wizards

Panels extend `WizardDescriptor.AbstractPanel<D>` and read/store a data object; `fireChange()` after validating re-evaluates *Next*/*Finish*:

```
WizardDescriptor<Data> w = new WizardDescriptor<Data>(new WizardDescriptor.ArrayIterator<Data>
(namePanel, physicsPanel), data);
DialogDisplayer.getDefault().notify(w).thenAccept(new Consumer<Object>() {
    @Override public void accept(Object option) { if (option == WizardDescriptor.FINISH_OPTION)
create(w.getData()); }
});
```

See `webbeans-demo/.../PlanetWizardAction.java` for the panels.



The demo's two-step planet wizard (`Planets ▸ New planet wizard…`): the step list on the left, the current panel's content, and Back / Next / Finish / Cancel whose enabled state follows `isValid()` of the panels. Dialogs float over the main window and are modal to the session.

6. Windows in depth

6.1 Lifecycle and state

```
@Override protected void componentOpened() { TaskList.get().addListener(this); tasksChanged(); }   //
subscribe
@Override protected void componentClosed() { TaskList.get().removeListener(this); }               //
unsubscribe
```

`componentOpened/Closed` bracket the time the window is in the dock; `componentShowing/Hidden` the time its tab is selected; `componentActivated/Deactivated` the keyboard focus. Subscribe to models and start timers in `componentOpened`, undo both in `componentClosed`.

`setModified(true)` marks the tab; the Workbench editor also bolds the name:

```
private void setDirty(boolean m) {
    if (dirty != m) {
        dirty = m;
        setHtmlDisplayName(m ? "<b>" + path.getFileName() + "</b>" : null);
        setModified(m);
    }
}
```

`makeBusy(true)` shows a spinner on the tab; `requestActive()` brings the window to the front; `setClosable(false)` removes the close button.

6.2 Where it docks: modes and your own

`mode = "explorer" | "navigator" | "editor" | "properties" | "output"` picks a standard mode. The Workbench's admin jar adds its own bottom mode and docks there:

```
@Mode.Registration(name = "admin", side = Mode.Side.BOTTOM, defaultSize = 180, position = 200)
@TopComponent.Registration(mode = "admin", openAtStartup = false, position = 100, roles = {"admin"})
@TopComponent.Description(preferredID = "ServerStatusTC", iconBase =
"org/webbeans/workbench/admin/icons/server.svg", persistenceType =
TopComponent.PERSISTENCE_ONLY_OPENED, helpId = "admin")
@TopComponent.OpenActionRegistration(displayName = "#CTL_OpenServerStatus", position = 950)
public final class ServerStatusTopComponent extends TopComponent implements ExplorerManager.Provider
{ … }
```

`roles = {"admin"}` hides the window *and* its Window-menu entry from users without the role (role checks happen on the server, per registry entry). `persistenceType` says whether the window is restored at the next login: `PERSISTENCE_ALWAYS` (default), `PERSISTENCE_ONLY_OPENED`, `PERSISTENCE_NEVER`. Users rearrange the dock by dragging; the layout is saved per user. A jar can ship a default layout per role: `@LayoutRegistration(role = "admin", resource = "org/webbeans/workbench/admin/layouts/admin.json")` (see `AdminLayout`). The platform's Help window uses the same mechanism for a `help` mode of its own on the right.

6.3 Many instances of one window: editors

A window without `@TopComponent.Registration` is created by your code — one per document. Give it a `Factory` so it is restored at the next login:

```
@TopComponent.Description(preferredID = "EditorTC", iconBase =
"org/webbeans/workbench/files/icons/text.svg", factory = EditorTopComponent.Factory.class, helpId =
"files.editor")
public final class EditorTopComponent extends TopComponent {

    public EditorTopComponent(Path path) { … }

    @Override
    protected void writeState(Map<String, Object> out) {
        out.put("path", path.toString());
    }

    /** Activates the editor of the file if one is open, else opens a new one. */
    public static EditorTopComponent openFor(WindowManager wm, Path path) {
        for (TopComponent tc : wm.getOpened()) {
            if (tc instanceof EditorTopComponent && ((EditorTopComponent) tc).path.equals(path)) {
                tc.requestActive();
                return (EditorTopComponent) tc;
            }
        }
        EditorTopComponent tc = new EditorTopComponent(path);
        tc.open();
        tc.requestActive();
        return tc;
    }

    public static final class Factory implements TopComponent.Factory {
        @Override
        public TopComponent create(Map<String, Object> state) {
            Object p = state.get("path");
            Path path = p == null ? null : Paths.get(String.valueOf(p));
            return path != null && Files.isRegularFile(path) ? new EditorTopComponent(path) : null;
// gone: skip it
        }
    }
}
```

6.4 The window’s lookup: how the rest of the platform sees your window

Every window has a `Lookup`; the *active* window’s lookup is the **global context** that context-sensitive actions, the Edit menu and the Properties window read. Put into it what your window “is about”:

```
associateLookup(Lookups.fixed(path, node, new Savable() {           // EditorTopComponent
    @Override public void save() throws IOException { doSave(); }
}));
```

Now *File* ▸ *Save* (a context action over `Savable`, §8) is enabled exactly while an editor with that lookup is active, and the Properties window shows the file’s node. Explorer windows do the same with `ExplorerUtils.createLookup(em, map)` (§7.3), and the dashboard swaps the clicked chart into an `InstanceContent` so *Export chart* applies to it:

```
private final InstanceContent content = new InstanceContent();
associateLookup(new AbstractLookup(content));
…
content.set(Collections.singleton(c), null);      // in the chart's click listener
```

Pattern — lookup in, actions out. Windows publish objects; actions declare which object they need. Neither knows the other’s class. This is what lets a *Files* window from one jar and a *Save* action from another cooperate, and what keeps `FileActions` free of any reference to the editor.

6a. Tables of plain objects

Not everything is a tree of nodes. For a grid of records — orders, log lines, query results — use `Table<T>` (`org.webbeans.ui.table`). The demo's *Orders* window shows 10 000 generated orders:

```
Table<Order> table = new Table<Order>();
table.setKeyFunction(new Function<Order, String>() { @Override public String apply(Order o) { return
String.valueOf(o.id); } });
table.addColumn("id", Bundle.COL_Id(), new Function<Order, Object>() { @Override public Object
apply(Order o) { return o.id; } }).width(70).align(Column.Align.RIGHT);
table.addColumn("customer", Bundle.COL_Customer(), new Function<Order, Object>() { @Override public
Object apply(Order o) { return o.customer; } })
     .editable(new BiConsumer<Order, String>() {
         @Override public void accept(Order o, String text) {
             if (text.trim().isEmpty()) throw new IllegalArgumentException(Bundle.ERR_Customer());
// shown on the cell
             o.customer = text.trim();
         }
     });
table.addColumn("date", Bundle.COL_Date(), new Function<Order, Object>() { @Override public Object
apply(Order o) { return o.date; } }).width(110);
table.addColumn("total", Bundle.COL_Total(), new Function<Order, Object>() { @Override public Object
apply(Order o) { return o.total; } }).align(Column.Align.RIGHT);
table.addColumn("status", Bundle.COL_Status(), new Function<Order, Object>() {
    @Override public Object apply(Order o) { return Cell.of(o.status.label(), o.status.icon()); }
});
ListTableModel<Order> model = new ListTableModel<Order>(orders);
table.setModel(model).setSelectionMode(Table.SelectionMode.MULTI).setShowFilter(true);
table.addSelectionListener(new Listener<Table.SelectionEvent>() {
    @Override public void onEvent(Table.SelectionEvent e) {
status.setText(Bundle.MSG_Selected(table.getSelected().size())); }
});
table.addRowMenuListener(new Listener<Table.RowEvent>() {
    @Override public void onEvent(Table.RowEvent e) { Actions.actionsToPopup(rowActions,
Lookups.fixed(table.getSelected().toArray()), e.getX(), e.getY()); }
});
```

What you get without more code: virtual scrolling (the browser asks for pages of 100 rows as you scroll), sortable headers, a filter box, click/Shift/Ctrl and keyboard selection, double-click (`addActivateListener`) and right-click events, inline editing on `editable` columns (double-click or F2; a thrown `IllegalArgumentException` flags the cell and keeps the old text), and numbers/dates formatted for the user's locale.

- **Keep the model in charge.** `model.update(order)` re-renders that row in place; `model.add(...)`, `remove(...)`, `set(...)` reset the table and the browser re-requests what it shows. Never hold row indices — rows are identified by the key function.
- **Big data lives elsewhere.** Extend `LazyTableModel<T>`: `count(query)` and `fetch(query, offset, limit)` run on a background thread (translate `query.getSortColumn()`, `isAscending()`, `getFilter()` into your SQL); the page is delivered on the session executor. Call `refresh()` when the data changed.
- **Cells are rendered on the server.** Return a value (formatted for the locale), a `Resource` (icon) or a `Cell` (`Cell.of(text, icon).style("ok")`, `Cell.html(...)`); the browser never sees your objects.
- For records that have actions, properties and drag and drop of their own, an `OutlineView` over nodes (§7) is still the better fit.

6a.1 Trees of plain objects: TreeTable

`TreeTable<T>` is the same table with a hierarchy in the first column (an `OutlineView` without nodes). Give it a `TreeTableModel<T>` — or `setTree(roots, childrenFn)` — and the columns you would give a `Table`; the expanded part is flattened on the server, so sorting (within siblings), the filter box (matches with their ancestors) and paging behave as before. `expand(node)`, `collapseAll()`, `expandAll()`, `structureChanged()` when children changed, `nodeChanged(node)` for one row; `addExpandListener` tells you what the user opened. The *Widgets* window groups the planets by type this way and feeds the selected planet into a `PropertyGrid`.

6a.2 Editing one object: PropertyGrid

`PropertyGrid<T>` shows one bean as name/value rows with typed editors — text, number, check box, date, a drop-down for enums (with their labels and icons) or for `choices(...)` — grouped by `category`, with an optional description pane. Declare each property with a getter and a setter; throw `IllegalArgumentException` from the setter to reject a value (the row shows the message and keeps the old value); listen to `addChangeListener` to persist. It is the Properties window for objects that are not nodes:

```
PropertyGrid<Planet> grid = new PropertyGrid<Planet>();
grid.addProperty("name", Bundle.PROP_Name(), String.class, getName,
setName).category(Bundle.CAT_Identity());
grid.addProperty("type", Bundle.PROP_Type(), Planet.Type.class, getType,
setType).category(Bundle.CAT_Identity());   // "Rocky", "Gas giant", … with icons
grid.addProperty("gravity", Bundle.PROP_Gravity(), Double.class, getGravity,
setGravity).category(Bundle.CAT_Physics());
grid.setBean(planet).setShowDescription(true);
```

6b. Text, code, boards and viewers

The tier-2 components live in the demo's *Editors* and *Boards* windows; all are plain components you add to a layout, and all keep the data on the server:

Need	Component	Notes
Edit source or configuration	`CodeEditor`	highlighting for Java, JS, JSON, XML, HTML, CSS, properties, Markdown, SQL, shell, Python, YAML; gutter `Annotation`s you set from the server; `goToLine`, cursor and gutter events
Edit rich text	`MarkdownEditor`	toolbar + live preview rendered by the server with the help pages' Markdown; the value is the Markdown source, `getHtml()` the sanitised HTML
Compare two texts	`DiffView`	side by side or inline; `setContext(n)` folds unchanged runs; `LineDiff.diff` / `unified` for your own use
Show a log or build output	`OutputView`	`append`/`error`/`warning` from any listener (batched per flush), ANSI colours, timestamps, follow-tail, link lines that report `LINK` (`append(text, style, "File.java:12")`)
A month or week of events	`Calendar`	keyed `Entry`s (all-day, timed, spanning, coloured); the user navigates (`RANGE` tells you the visible period), double-clicks days, drags entries → `MOVE` (you apply it: `cal.addEntry(e.getMoved())`)
A board of cards	`Kanban`	columns with WIP limits and add buttons; drag between columns is applied on the server and reported as `MOVE`
Boxes and arrows	`Diagram` + `Palette`	SVG canvas with pan/zoom, drag (applied, reported), selection, Delete, connect by dragging a port, drops from a `Palette` (`DROP` → you add the node); `autoLayout()`
Look at a picture or a PDF	`ImageViewer`, `PdfViewer`	zoom/pan/rotate; the browser's PDF viewer over bytes served from the session (`setDocument(bytes, name)`)

Two rules they share with the table: the browser only ever sees keys and rendered text, and a user gesture that changes data (a drop, a move, a connection) arrives as an event on the session thread — some components apply it to their own model first (`Kanban`, `Diagram` node positions), the ones whose model is yours (`Calendar` dates, `Diagram` edges and drops, `OutputView` links) only report it.

7. Nodes and explorer views

Trees, lists and tables of *things* are built from **nodes**. A `Node` has a name, an icon, children, a lookup (the thing it represents), properties (for columns and the Properties window) and actions (its popup menu). The views — `TreeView`, `OutlineView`, `ListView`, `PropertySheet` — render nodes and talk to an `ExplorerManager` that holds the root and the selection.

7.1 A node per object

```
public final class TaskNode extends AbstractNode {

    public static final String ACTIONS_FOLDER = "Tasks/Actions";

    private final Task task;

    public TaskNode(Task task) {
        super(Children.LEAF, Lookups.singleton(task));          // the Task is in the lookup → context actions see it
        this.task = task;
        setName(task.getTitle());
        setIconBase("org/webbeans/tutorial/" + (task.isDone() ? "done.svg" : "task.svg"));
        setCanRename(true);
        setCanDestroy(true);
    }

    @Override
    public Action[] getActions(boolean context) {               // the popup: whatever was registered under Tasks/Actions
        List<Action> all = Actions.forPath(ACTIONS_FOLDER);
        return all.toArray(new Action[all.size()]);
    }

    @Override
    public void setName(String n) {                             // in-place rename in the view goes through the model
        if (task != null && !task.getTitle().equals(n)) {
            TaskList.get().rename(task, n);
        }
        super.setName(n);
    }

    @Override
    public void destroy() throws IOException {                  // Edit ▸ Delete / the Delete key
        TaskList.get().remove(task);
        super.destroy();
    }

    @Override
    public String getHtmlDisplayName() {
        return task.isDone() ? "<s>" + task.getTitle() + "</s>" : null;
    }

    @Override
    protected Sheet createSheet() {                             // properties: columns in an OutlineView, rows in the Properties window
        Sheet sheet = new Sheet();
        Sheet.Set set = Sheet.createPropertiesSet();
        set.put(new PropertySupport.Name(this, Bundle.PROP_Title(), Bundle.HINT_Title()));
        set.put(new PropertySupport.ReadWrite<Boolean>("done", Boolean.class, Bundle.PROP_Done(), Bundle.HINT_Done()) {
            @Override public Boolean getValue() { return task.isDone(); }
            @Override public void setValue(Boolean v) { TaskList.get().setDone(task, Boolean.TRUE.equals(v)); }
        });
        sheet.put(set);
        return sheet;
    }
}
```

A `ReadWrite` property is editable in place (a check box for `Boolean`, a text field for `String`, a drop-down for enums — labelled and iconed through `EnumLabels`, §2.6; `PropertyEditors` registers editors per type, `PropertyEditors.registerEnum` overrides one enum's labels and icons). Throw `IllegalArgumentException(message)` from `setValue` or `setName` to reject a value — the view shows the

message (see `PartNode`'s colour property and `FileNode.setName`). To draw a value with an icon, colour or tooltip while keeping it editable as text, give the property a renderer: `property.setRenderer(new Function<Type, Cell>() { … return Cell.of(label, icon).style("wb-muted"); })` — `OutlineView` cells and the Properties window both honour it (`PlanetNode`'s type property in the demo).

7.2 Children

Two ways to produce children. **Keys** when you have the list in memory and want cheap diffs:

```
static final class TaskChildren extends Children.Keys<Task> {
    void setTasks(List<Task> tasks) {
        setKeys(tasks);                 // adds/removes nodes to match; unchanged keys keep their
nodes
        for (Task t : tasks) {
            refreshKey(t);              // title or done may have changed: recreate that node
        }
    }

    @Override
    protected Node[] createNodes(Task key) {
        return new Node[] {new TaskNode(key)};
    }
}
```

A **ChildFactory** when computing the list is slow (I/O): `Children.create(factory, true)` runs `createKeys` on a background thread and shows a *Please wait...* node meanwhile:

```
final class FolderChildFactory extends ChildFactory<Path> {                    // Workbench files
    @Override
    protected boolean createKeys(List<Path> keys) {
        try (DirectoryStream<Path> ds = Files.newDirectoryStream(dir)) { … keys.addAll(dirs);
keys.addAll(files); }
        return true;                                                             // false = call me
again
    }
    @Override
    protected Node createNodeForKey(Path key) { return new FileNode(key); }
}
// in FileNode: super(Files.isDirectory(path) ? Children.create(new FolderChildFactory(path), true) :
Children.LEAF, lookupFor(path));
```

Call `factory.refresh(false)` when the directory changed.

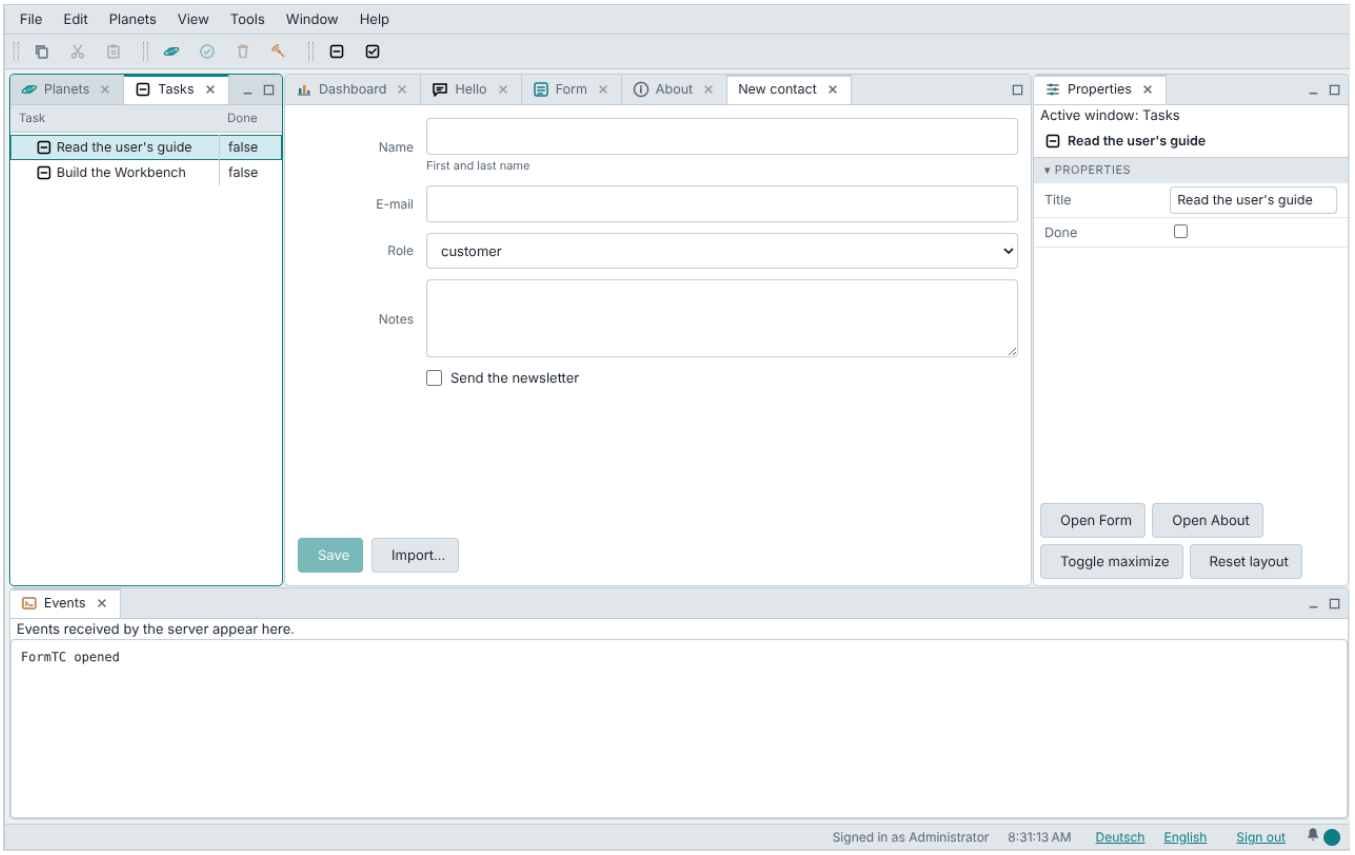
7.3 The explorer window

```
public final class TasksTopComponent extends TopComponent implements ExplorerManager.Provider,
TaskList.Listener {

    private final ExplorerManager em = new ExplorerManager();
    private final ActionMap map = new ActionMap();
    private final OutlineView view = new OutlineView(Bundle.COL_Title());
    private final TaskChildren children = new TaskChildren();

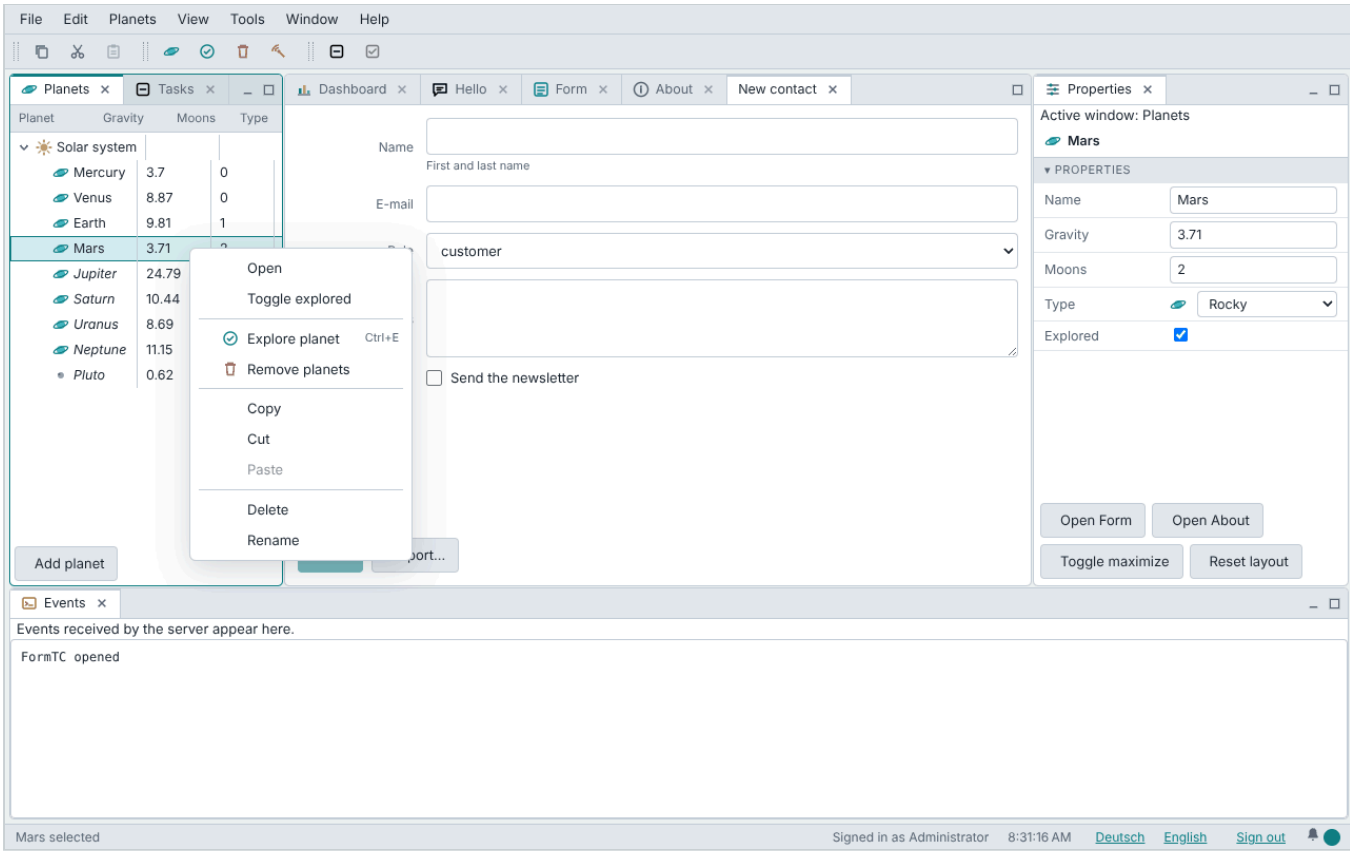
    public TasksTopComponent() {
        setDisplayName(Bundle.CTL_TasksTC());
        setLayout(new Layout.Border());
        ExplorerUtils.activateActions(em, map, true);          // copy/cut/paste/delete follow the
selection
        associateLookup(ExplorerUtils.createLookup(em, map));  // selected nodes (+ their Tasks) in
the window's lookup
        view.setPropertyColumns("done", Bundle.COL_Done());    // tree column + the "done" property
as a column
        view.setColumnWidth("done", 60);
        view.setRootVisible(false);
        view.setTestId("tasks");
        AbstractNode root = new AbstractNode(children);
        root.setName("tasks");
        em.setRootContext(root);
        add(view, Layout.Border.CENTER);
    }

    @Override public ExplorerManager getExplorerManager() { return em; }
    @Override protected void componentOpened() { TaskList.get().addListener(this); tasksChanged(); }
    @Override protected void componentClosed() { TaskList.get().removeListener(this); }
    @Override public void tasksChanged() { children.setTasks(TaskList.get().getTasks()); }
}
```



`TasksTopComponent` in the explorer mode: an `OutlineView` with the tree column and the `done` property column. The selected node's properties appear in the Properties window on the right — the window system passes the explorer's selection on through the activated-nodes context (§6.4).

The views find the manager through `ExplorerManager.Provider` on the enclosing window. Swap `OutlineView` for `TreeView` and you have a plain tree (`PartsTopComponent`); `view.setPropertyColumns("size", …, "modified", …)` plus `setSortable(true)` turns `FilesTopComponent` into a sortable file table. Right-click shows `getActions`, double-click runs the node's preferred action (by default: `open()` on an `Openable` in the lookup, as the Workbench's file nodes do; or `setPreferredAction(...)`), F2 renames, Delete destroys — none of which you wrote. `tree.expandNode(n)` expands from code; the view remembers what is expanded (by the user or by you) and restores it after a reconnect or a root change — the Help window's table of contents relies on that.



The demo's Planets window with the right-click menu of the Mars node: the node's own `getActions` entries first (`Open`, `Toggle explored`), then the actions registered for the `Planets/Actions` path (`Explore planet`, `Remove planets`), then the standard copy/cut/paste/delete/rename provided by `ExplorerUtils.activateActions`.

Pattern — model → nodes → views, never views → views. The 3D viewer and the Parts tree of the Workbench both show `PartsModel`; selecting in one selects in the other because both listen to the *model's* selection (`PartsModel.setSelected`), not to each other.

8. Actions, menus, toolbars, shortcuts and popups

An action is a class with three annotations: identity, presentation, and the places it appears. The tutorial has one of each kind:

```
/** Always enabled: no constructor parameter. In the Tools menu, the Tasks toolbar, Ctrl+Shift+T and
the node popup. */
@ActionID(category = "Tasks", id = "org.webbeans.tutorial.AddTaskAction")
@ActionRegistration(displayName = "#CTL_AddTask", iconBase = "org/webbeans/tutorial/task.svg")
@ActionReferences({
    @ActionReference(path = "Menu/Tools", position = 400),
    @ActionReference(path = "Toolbars/Tasks", position = 100),
    @ActionReference(path = "Shortcuts", name = "DS-T"),
    @ActionReference(path = TaskNode.ACTIONS_FOLDER, position = 200)})
public static final class AddTaskAction implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        final NotifyDescriptor.InputLine in = new NotifyDescriptor.InputLine(Bundle.LBL_Title(),
Bundle.TITLE_AddTask());
        DialogDisplayer.getDefault().notify(in).thenAccept(new Consumer<Object>() {
            @Override
            public void accept(Object answer) {
                String title = in.getInputText() == null ? "" : in.getInputText().trim();
                if (answer == NotifyDescriptor.OK_OPTION && !title.isEmpty()) {
                    TaskList.get().add(title);
                }
            }
        });
    }
}

/** Context action: enabled while the global context holds at least one Task (the selection of the
Tasks window). */
@ActionID(category = "Tasks", id = "org.webbeans.tutorial.DoneAction")
@ActionRegistration(displayName = "#CTL_Done", iconBase = "org/webbeans/tutorial/done.svg")
@ActionReferences({
    @ActionReference(path = "Menu/Tools", position = 410),
    @ActionReference(path = "Toolbars/Tasks", position = 110),
    @ActionReference(path = TaskNode.ACTIONS_FOLDER, position = 100)})
public static final class DoneAction implements ActionListener {
    private final List<Task> tasks;

    public DoneAction(List<Task> tasks) {           // List<T>: any number selected; T alone: exactly
one
        this.tasks = tasks;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        for (Task t : tasks) {
            TaskList.get().setDone(t, true);
        }
        StatusDisplayer.getDefault().setStatusText(Bundle.MSG_Done(tasks.size()));
    }
}
```

The rules:

- **No-arg constructor → always enabled. Constructor with `T` → enabled when the global context has exactly one `T`; `List<T>` → one or more.** The platform instantiates the class on invocation with the objects from the context. `surviveFocusChange = true` keeps the last context

while a non-context window (a toolbar, a dialog) has focus — the Workbench *Save* uses it so Ctrl+S works after clicking the toolbar.

- `@ActionReference(path = "Menu/<menu>")`, `"Toolbars/<toolbar>"`, `"Shortcuts"` and any folder your nodes read with `Actions.forPath(...)`. `position` orders, `separatorBefore/After` draw separators. Shortcut names follow NetBeans: `D-S` (Ctrl/Cmd+S), `DS-T` (Ctrl+Shift+T), `F5`, `F1`.
- A toolbar or menu is a registry folder; declare it once with `@FolderRegistration(path = "Toolbars/Tasks", displayName = "#CTL_TasksToolbar", position = 500)` (on any class — `TaskActions` carries it). The standard menus are `Menu/File, Edit, View, Tools, Window, Help`.
- `roles = {"admin"}` on `@ActionRegistration` hides the action from other users (`EndSessionAction`). `asynchronous = true` runs `actionPerformed` off the session thread with a progress indicator (the demo's `ScanAction`).

Invoking an action from code: `Actions.forID("Tasks", "org.webbeans.tutorial.AddTaskAction")` returns the action; `Actions.invoke(action, source, context)` runs it (tests do this). `Actions.forPath("Tasks/Actions")` returns the folder's actions for a popup.

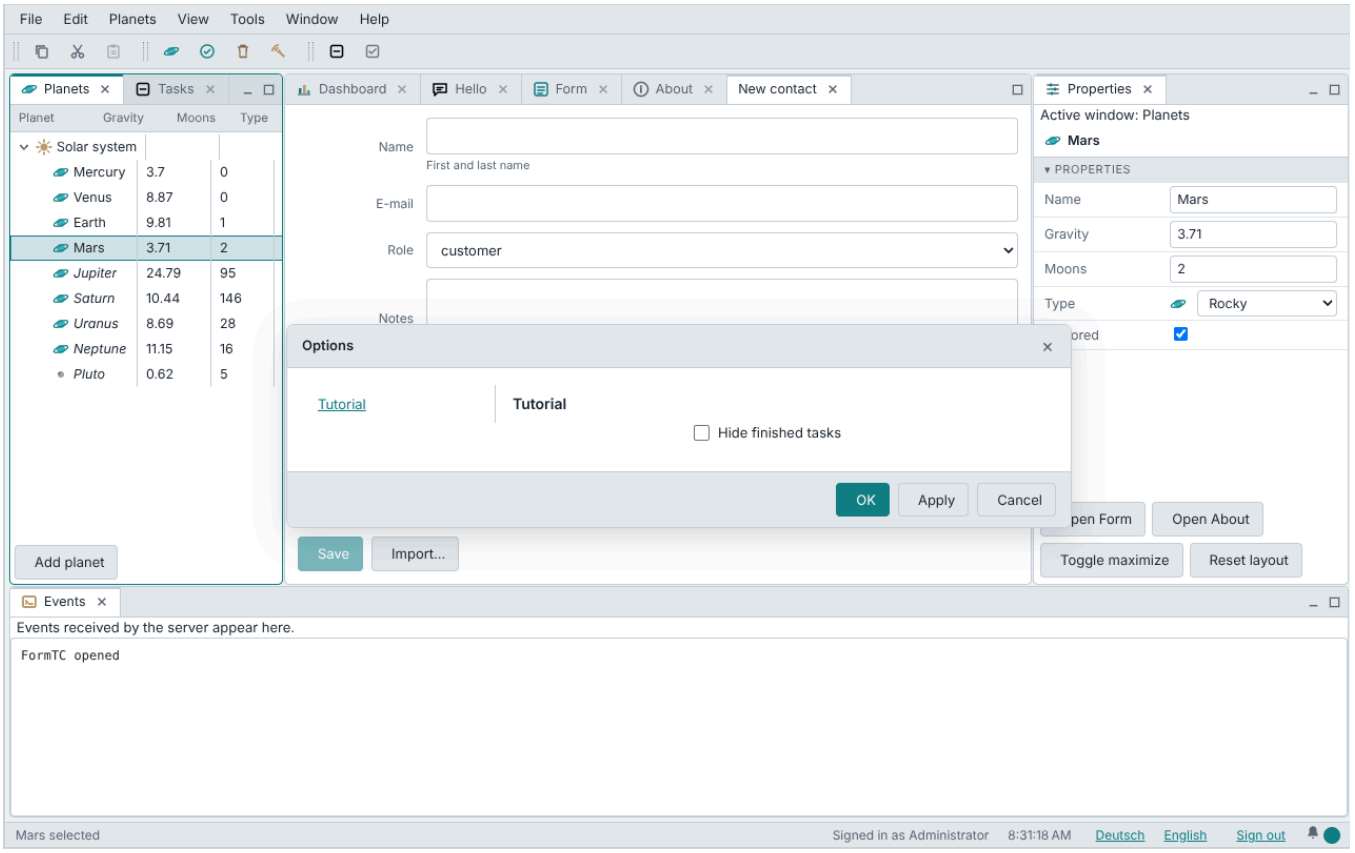
Check-box actions backed by a preference: `Actions.checkbox(ViewerActions.class, "spin", true, onToggle, displayName)` (Workbench *View* ▸ *Spin rotor*). **Window-local actions** (the Output window's *Edit* ▸ *Delete* = clear): put an `ActionMap` with the key into the window's lookup — `map.put(ActionMap.DELETE, new AbstractAction("clear") {...}); associateLookup(Lookups.singleton(map));` — and the Edit menu's callback action finds it while the window is active.

Quick Search (Ctrl+I): every menu action, open window and help topic is reachable by typing part of its name (or its initials: "ot" → *Open Toolbars*). Contribute your own results with a `QuickSearchProvider` registered under `QuickSearch/Providers` — `getCategory()` for the heading and `search(query, response)` adding `(displayName, detail, icon, Runnable)` results — the Workbench could offer files and parts this way.

Pattern — register in folders, not on windows. `TaskNode` asks `Actions.forPath("Tasks/Actions")`; it does not list its actions. Any jar can add a popup entry without touching the node — the Workbench *Files* popup, *Parts* popup and *Scene3D* viewport toolbar all work this way.

9. Options and preferences

Per-user settings use `java.util.prefs.Preferences` obtained from `WbPreferences.forModule(YourClass.class)` (stored in the user's data directory, isolated per module). A page in *Tools ▸ Options* is an `OptionsPanelController`:



`Tools ▸ Options` with the tutorial's page. Every registered `OptionsPanelController` becomes a category on the left; OK / Apply / Cancel call `applyChanges()` / `cancel()` on all of them.

```
@OptionsPanelController.TopLevelRegistration(categoryName = "#CTL_TutorialOptions", position = 900)
@Messages({"CTL_TutorialOptions=Tutorial", "LBL_HideDone=Hide finished tasks"})
public final class TutorialOptions extends OptionsPanelController {

    static final String PREF_HIDE_DONE = "hideDone";

    private final CheckBox hideDone = new CheckBox(Bundle.LBL_HideDone());
    private Preferences prefs;

    @Override
    public Component getComponent(Lookup master) {               // once per opening of the dialog
        prefs = WbPreferences.forModule(TutorialOptions.class);
        hideDone.setTestId("opt-hide-done");
        hideDone.addChangeListener(new Listener<ChangeEvent<Boolean>>() {
            @Override
            public void onEvent(ChangeEvent<Boolean> e) {
                changed();                                       // lets the dialog enable Apply
            }
        });
        Panel p = new Panel(new Layout.Form());
        p.addRow("", hideDone);
        return p;
    }

    @Override public void update() { hideDone.setSelected(prefs.getBoolean(PREF_HIDE_DONE, false)); }
// load
    @Override public void applyChanges() { prefs.putBoolean(PREF_HIDE_DONE, hideDone.isSelected()); }
// OK / Apply
    @Override public void cancel() { }
    @Override public boolean isValid() { return true; }
    @Override public boolean isChanged() { return prefs.getBoolean(PREF_HIDE_DONE, false) !=
hideDone.isSelected(); }
}
```

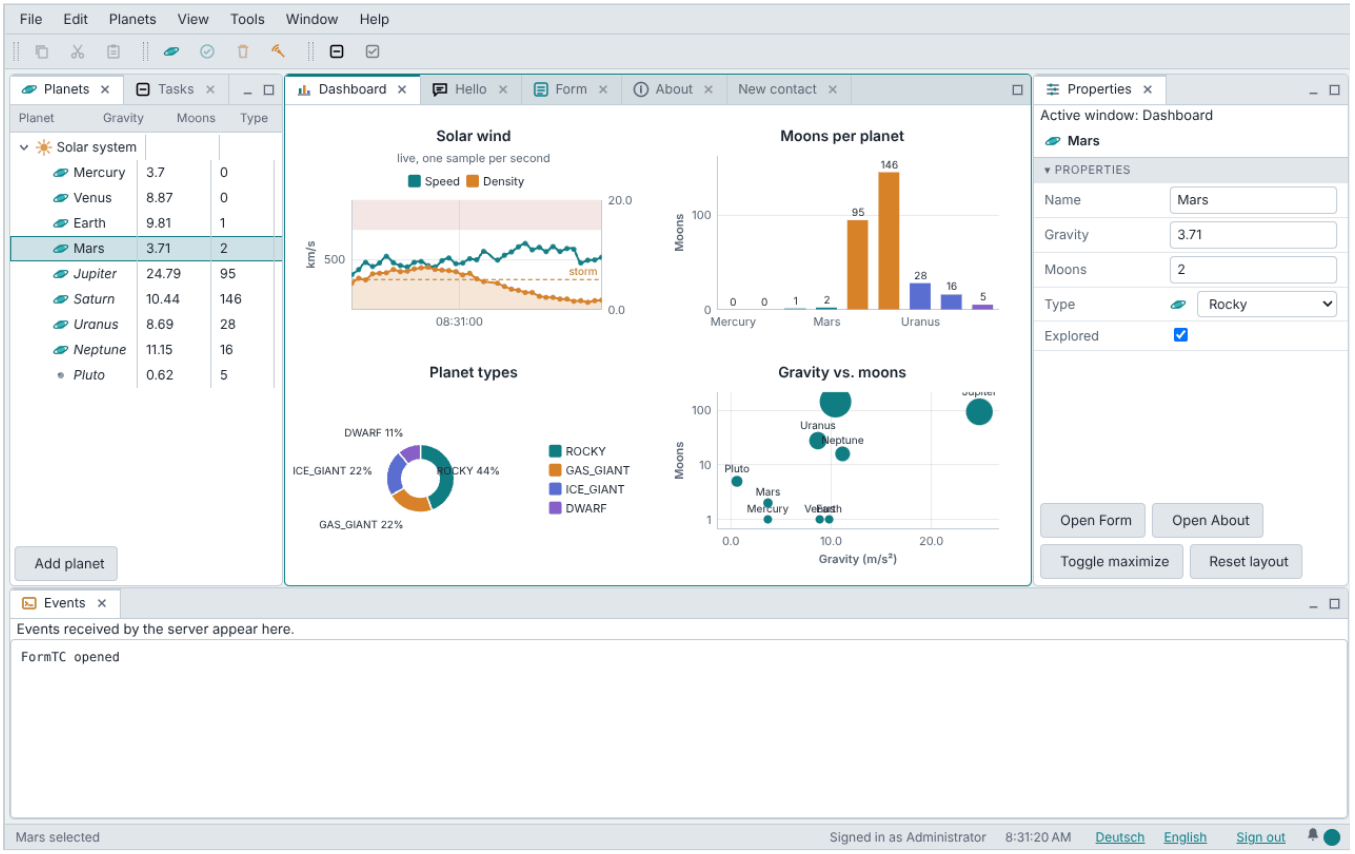
The dialog lists every registered category (filtered by `roles`) in a strip on the left, shows the pages in a `Layout.Stack`, and calls `update()` before showing, `applyChanges()` on OK/Apply, `cancel()` otherwise. Validation: `DashboardOptions` validates two text fields with ranges and returns `isValid()` accordingly — the OK button follows. Override `getHelpId()` to return a help page id and the dialog's *Help* button opens it while your category is selected (`DashboardOptions` returns `"dashboard.options"`). Open the dialog from code with `OptionsDisplayer.getDefault().open("TutorialOptions")`.

Pattern — apply settings where they are used, at the moments they matter. `DashboardOptions.applyChanges()` calls `DashboardTopComponent.settingsChanged()`, which restarts the timers of the open dashboards; `WorkbenchOptions` applies theme and language to the session immediately *and* a `MainWindowDecorator` (`SessionSetup`) re-applies them when the user logs in next time:

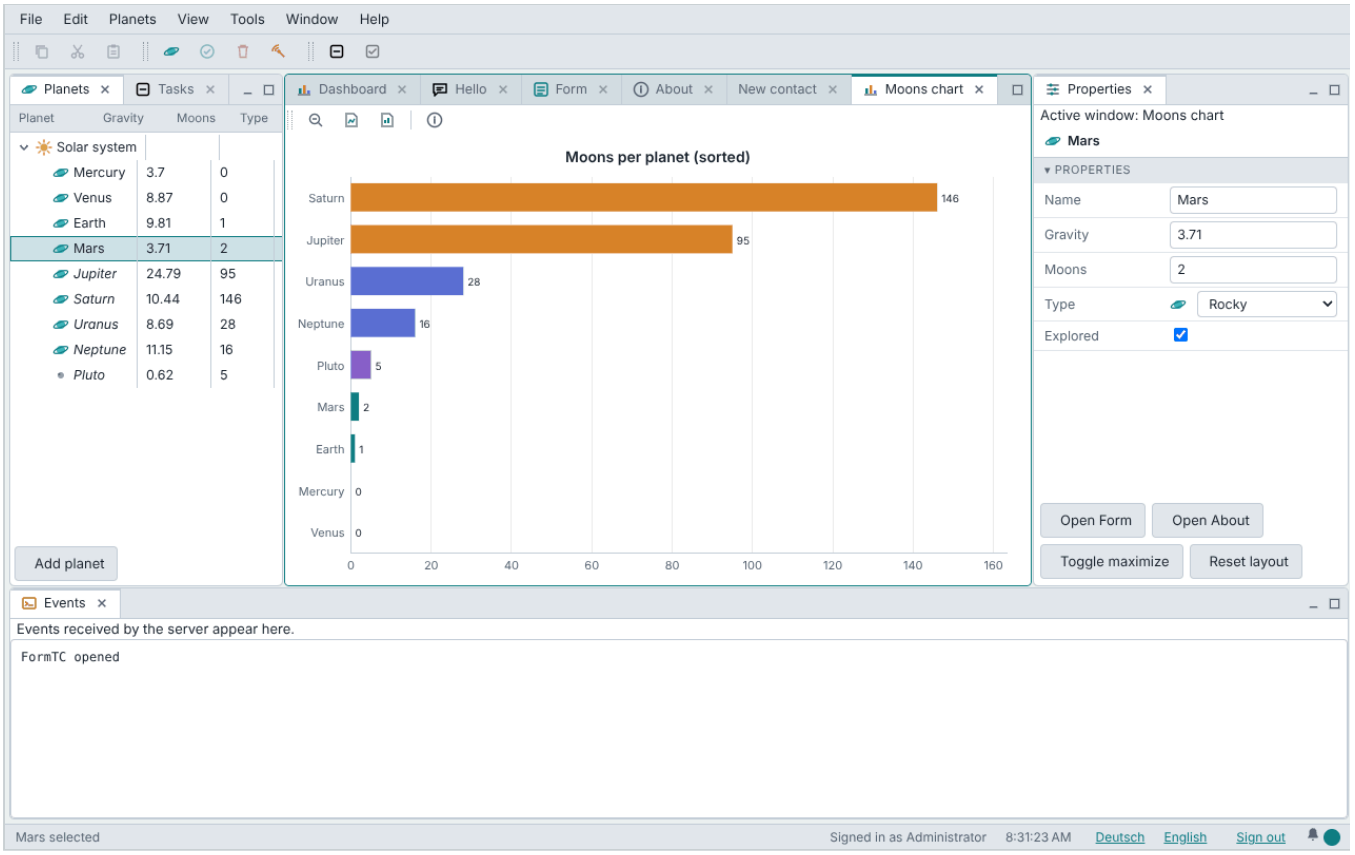
```
@ServiceProvider(service = MainWindowDecorator.class, position = 5000)
public final class SessionSetup implements MainWindowDecorator {
    @Override
    public void decorate(UI ui, MainWindow window) {
        Preferences prefs = WbPreferences.forModule(ui.getSession(), WorkbenchOptions.class);
        String theme = prefs.get(WorkbenchOptions.PREF_THEME, null);
        if (theme != null) {
            ui.getRoot().set(UI.Root.THEME, theme);
        }
        …
    }
}
```

10. Charts

A `Chart` has a type, axes, a legend and a model of series; points are appended or replaced and only the difference travels to the browser. The Workbench dashboard:



The demo's Dashboard window: a 2×2 `Layout.Grid` of charts — a live line chart with two axes and a shaded "storm" band, a bar chart, a donut and a bubble chart on a log axis. The line chart gets one appended point per second; the rest of the page is untouched.



A single chart in a `ChartTopComponent` (`Window ▸ Moons chart`): the toolbar has reset-zoom, save-as-SVG and save-as-PNG plus whatever is registered under `Charts/Actions` (the demo adds an info action); horizontal bars sorted by value with labels at the bar ends.

```
private final Chart cpu = new Chart(ChartType.LINE);
private final Chart requests = new Chart(ChartType.BAR);
private final Chart traffic = new Chart(ChartType.DONUT);

cpu.setTitle(Bundle.CHART_Cpu()).setTooltip(TooltipMode.SHARED).setZoom(ZoomMode.X).setTestId("chart-
cpu");
cpu.getXAxis().setType(AxisType.TIME).setFormat("HH:mm:ss");
cpu.getYAxis().setMin(0.0).setMax(100.0).setFormat("#0'%'").addBand(Bundle.BAND_Hot(), 90, 100,
"#b43b3b");
cpu.getLegend().setVisible(false);
cpu.getModel().addSeries("cpu", Bundle.SER_Cpu());
requests.setTitle(Bundle.CHART_Requests()).setLabels(true).setTestId("chart-requests");
requests.getModel().setCategories(Arrays.asList("/api", "/static", "/ws", "/auth"));
requests.getModel().addSeries("req", Bundle.SER_Requests());
traffic.setTitle(Bundle.CHART_Traffic()).setTestId("chart-traffic");
traffic.getModel().addSeries("share", Bundle.SER_Traffic());

Panel grid = new Panel(new Layout.Grid(2, 2, 8, 8).fill());
grid.setPadding(8);
grid.add(cpu); grid.add(requests); grid.add(traffic);
```

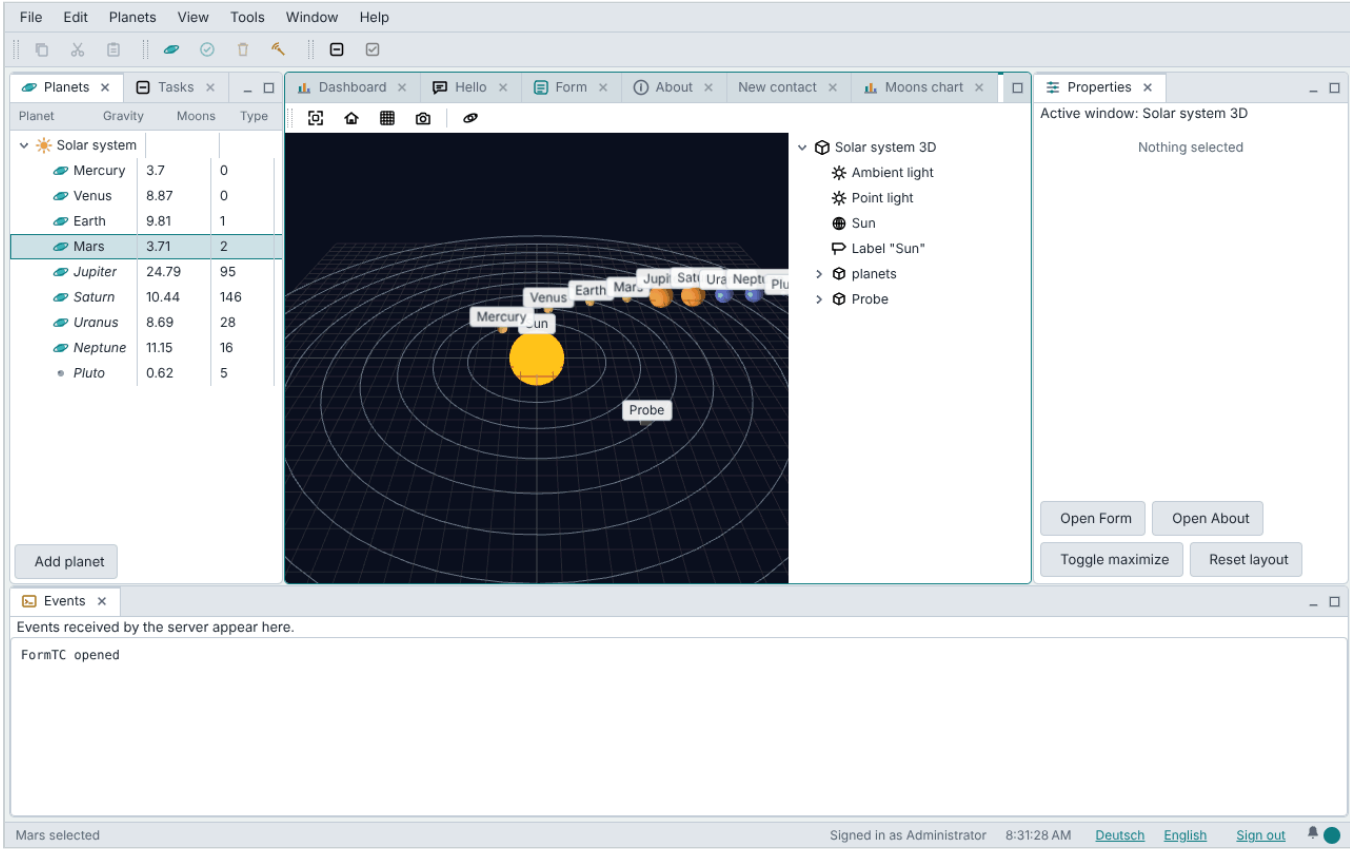
Feeding it from a timer (§4): a sliding window for the time series, full replacement for the bars and the donut (keyed points so the browser animates instead of redrawing):

```
cpu.getModel().getSeries("cpu").setWindow(120);
…
cpu.getModel().getSeries("cpu").addPoint(Point.of(Instant.now(), load));
List<Point> req = new ArrayList<Point>();
for (Map.Entry<String, Double> e : s.requestsByPath().entrySet()) {
    req.add(Point.of(e.getKey(), Math.round(e.getValue())).withKey(e.getKey()));
}
requests.getModel().getSeries("req").setPoints(req);
```

Interaction: `addPointClickListener` (the dashboard puts the clicked chart into its lookup so *Export chart as PNG* — a context action over `Chart` — applies to it), `addRangeSelectListener` with `ZoomMode.X`, `addLegendToggleListener`. Export: `chart.download("png", "cpu.png")` sends the file to the browser; `chart.exportPng(1200, 700)` resolves with the bytes on the server (rendered by the browser on the round trip — complete it with `whenComplete`, you are on another thread). Series styling: `setColor`, `setLineStyle`, `setMarker`, `setSmooth`, `setStack`, a second axis with `setYAxis(1)`; mixed types with `addSeries(id, name, ChartType.LINE)` on a bar chart.

11. 3D scenes

`Scene3D` is a component whose content is a scene graph of `Node3D`s (groups, meshes, lines, points, labels, lights) with a camera and orbit controls; the browser renders it with three.js and reports picks, hovers, drags and camera moves. `Scene3DTopComponent` wraps it with the viewport toolbar. The Workbench viewer builds a turbine:



The demo's `Solar system 3D` window: a `Scene3DTopComponent` with the viewport toolbar (fit, reset view, toggle grid, snapshot, plus the actions registered under `Scene3D/Actions`), the orbit camera, labelled spheres on ring orbits and the scene-graph tree on the right. The tree is an ordinary explorer view over `Node3D` nodes, so selection in either side selects in the other.

```
public ViewerTopComponent() {
    super(new Scene3D());
    final Scene3D scene = getScene();
    scene.setGrid(20, 20).setShadows(true).setHoverEffect(HoverEffect.HIGHLIGHT);
    scene.getCamera().setPosition(7, 5, 9).lookAt(0, 0, -1);
    scene.getControls().setDistanceLimits(3, 60).setPolarLimits(0.05, Math.PI / 2 + 0.2);
    DirectionalLight sun = new DirectionalLight(0xffffff, 0.9);
    sun.setShadowSize(12);
    sun.setPosition(8, 14, 10).setCastShadow(true);
    scene.getRoot().add(new AmbientLight(0xffffff, 0.45), sun);
    Mesh floor = new Mesh("floor", Geometry.plane(30, 30),
Material.standard(0x232a33).setRoughness(1));
    floor.setRotationEuler(-Math.PI / 2, 0, 0).setPosition(0, -4,
0).setReceiveShadow(true).setPickable(false);
    scene.getRoot().add(floor, fixed, rotor);
    scene.addListener(Scene3D.PICK, new Listener<PickEvent>() {
        @Override
        public void onEvent(PickEvent e) {
            Object data = e.getUserData();
            Part p = data instanceof Part ? (Part) data : null;
            model.setSelected(p);                                    // the model's selection
drives tree + scene
            if (p != null && e.isDoubleClick()) {
                scene.fit(e.getNode());
            }
        }
    });
}
```

Pattern — bind a collection to the scene with `SceneBinder`. Give it the group to fill, a key function, a creator and an updater; then `setItems(parts)` after a change and `refresh(part)` for one item — nodes are created, updated or removed to match, exactly like `Children.Keys` for nodes:

```
binder = new SceneBinder<Part>(fixed, new Function<Part, Object>() {
    @Override public Object apply(Part p) { return p.getId(); }
}, new Function<Part, Node3D>() {
    @Override public Node3D apply(Part p) { return create(p); }
}, new BiConsumer<Part, Node3D>() {
    @Override public void accept(Part p, Node3D n) { update(p, n); }
});

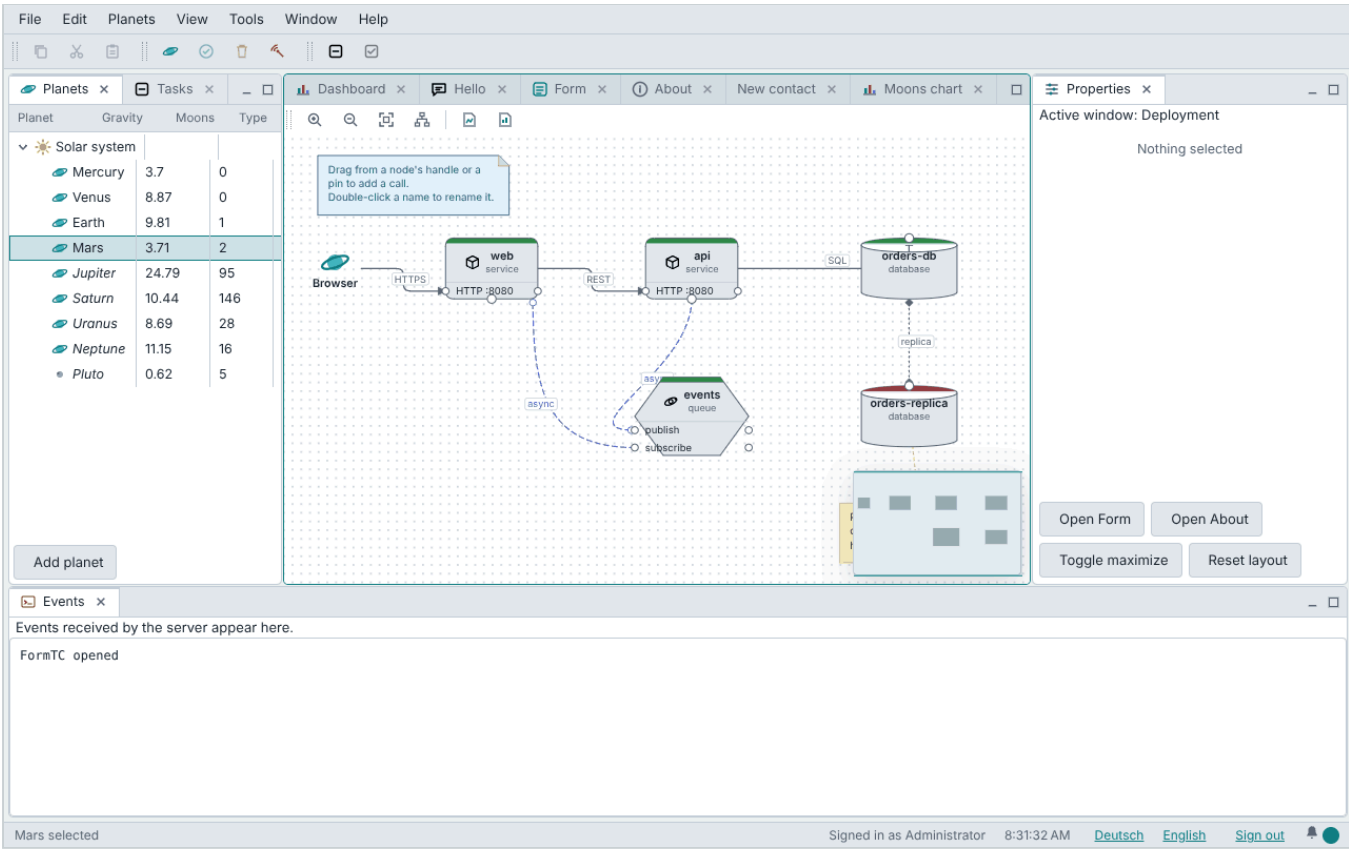
private Node3D create(Part p) {
    Group g = new Group(p.getName());
    Mesh body = new Mesh(p.getName(), Geometries.BLADE,
Material.standard(p.getColor()).setSide(Material.Side.DOUBLE).setRoughness(0.4).setMetalness(0.6));
    body.setCastShadow(true).setUserData(p);                         // what a pick
reports
    g.addChild(body);
    if (p.getKind() == Part.Kind.SENSOR) {
        g.addChild(new Label3D(p.getName())).setPosition(0, 0.5, 0);
    }
    g.setUserData(p);
    return g;
}
```

Animation runs in the browser: `rotor.animate(Animation.rotate(Axis.Z, 1.2))`, `rotor.stopAnimations()`. Selection highlight: `scene.setSelection(Collections.singletonList(node))`. Draggable objects: `probe.setDraggable(DragConstraint.plane(Axis.Y, 0).snap(0.5))` and a `DRAG` listener that may `e.reject()` the end position (the demo’s solar system). Your own geometry goes through `Geometry.buffer()` (`Geometries.BLADE`), sent once as a blob; glTF files through

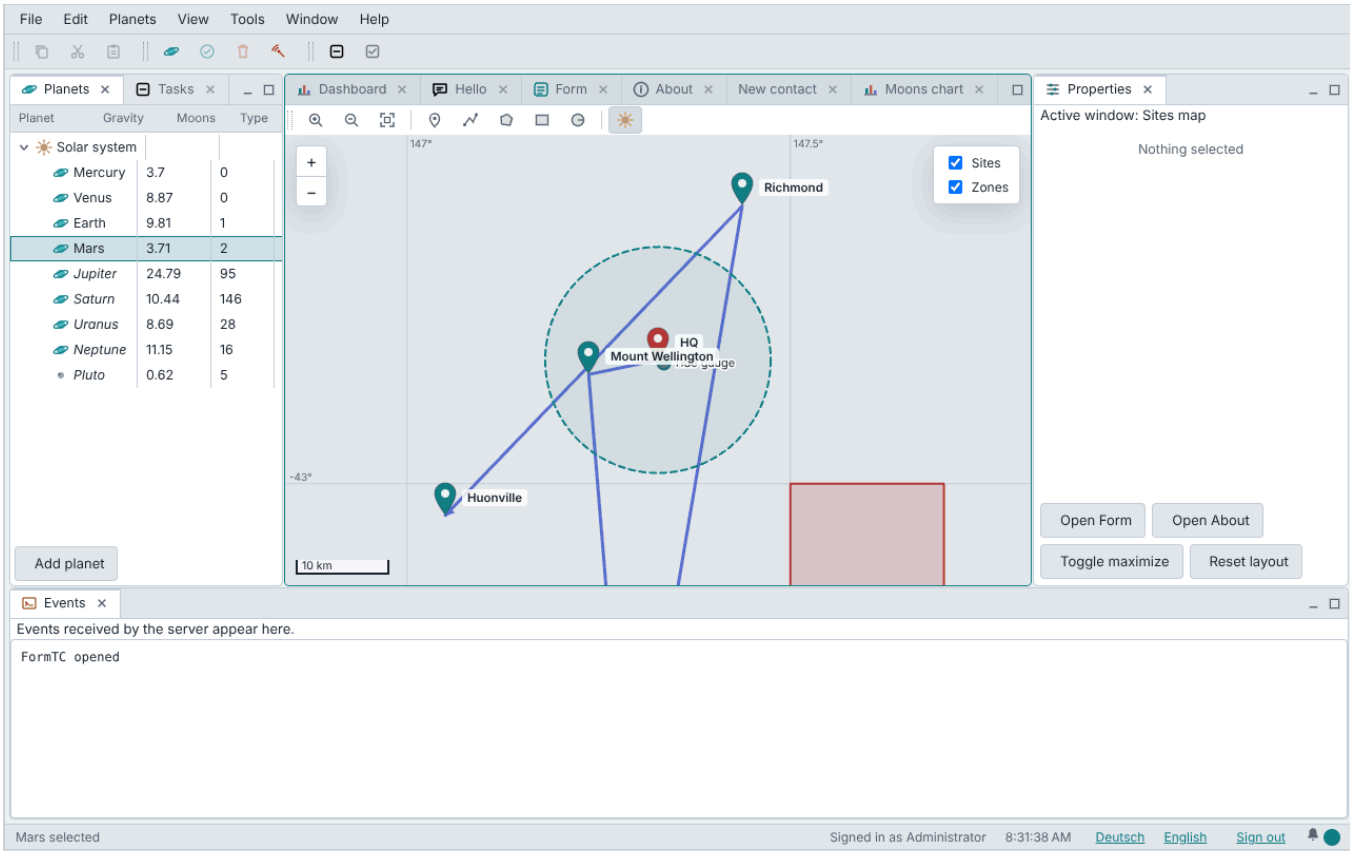
`Gltf.load(stream)`. Expose the scene graph as nodes (`Scene3DChildren`) to get a tree and property editing for free. Snapshots: `scene.snapshot(1280, 720)` (bytes), the toolbar's *Save PNG*, or `SnapshotAction` in the Workbench.

11a. Graphs and maps

Two more windows of the demo show the remaining “big” components; both are documented in full in the Component Guide (§7a `GraphScene`, §7b `MapView`), so this is only a look at what they give you.



`Window ▸ Deployment graph`: a `GraphTopComponent` over a `GraphScene` — nodes with shapes, pins and an accent bar, edges with orthogonal, curved and direct routers and different anchor shapes, a note, the minimap, the dotted grid, and a toolbar with zoom, fit, auto-layout (`TreeGraphLayout`), save as SVG/PNG and the actions registered under `Graph/Actions`. Users drag nodes, draw edges from a pin or a node handle, rename by double-click and get a popup per node; every change arrives as an event on the server (`webbeans-demo/.../DeploymentTopComponent.java`).



`Window ▸ Sites map`: a `MapTopComponent` over a `MapView` with markers, a polyline, a circle and a rectangle on two toggleable layers (`Sites`, `Zones`), the zoom control, the scale bar and a graticule. The base map is a `TileSource` (OpenStreetMap, proxied and cached by the server; shown here with the base map switched off, which is what you also get without internet access). The toolbar has zoom, fit, the draw tools for markers, lines, polygons, rectangles and circles, and the actions registered under `Map/Actions` (`webbeans-demo/.../SitesMapTopComponent.java`).

12. Services, sessions and server push

12.1 Three scopes

Scope	How to declare	How to get it
Global (one per JVM)	`@ServiceProvider(service = MetricsSampler.class)`	`Lookup.getDefault().lookup(MetricsSampler.class)`
Session (one per logged-in user)	`@Registration(path = "Services/Session", type = TaskList.class)`	`UISession.current().getLookup().lookup(TaskList.class)`
Tab (one per browser tab)	the `UI`: `UI.current()`, `getUI()` in a component	`ui.getLookup()`, `ui.addToLookup(o)`

`TaskList` (tutorial) and `PartsModel` (Workbench) are session services: every window of the user, in every tab, shares them, and two users never see each other's. A session service implementing `AutoCloseable` is closed when the session ends (`PartsModel` stops its simulation there).

12.2 The session bus: publish/subscribe across windows and tabs

```
// publisher (anywhere): the Workbench's Log helper
UISession.current().publish(new LogMessage(LogMessage.Level.INFO, "files", "Saved README.md"));

// subscriber: the Output window, in componentOpened
subscription = getUI().getSession().subscribe(LogMessage.class, new Consumer<LogMessage>() {
    @Override public void accept(LogMessage m) { append(m); }
});
// … and subscription.remove() in componentClosed
```

`LogMessage` is the only class the Workbench jars share (it lives in `workbench-core`, package `org.webbeans.workbench.core.api`, which the other projects depend on). The Output window of *every* tab of the user shows the line — the subscriber runs on the session thread, for each UI.

12.3 Push from a background thread

A server-side event (a sensor reading, a message queue, a finished job) reaches the user by hopping onto the session executor. `PartsModel` heats its sensors on a daemon thread:

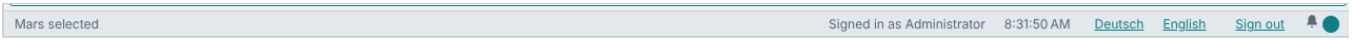
```
/** Runs on the simulation thread: computes, then publishes on the session executor. */
private void step() {
    final List<Part> changed = new ArrayList<Part>();
    synchronized (this) {
        for (Part p : parts) {
            if (p.getKind() == Part.Kind.SENSOR) {
                … // heat or cool the sensor, flip p.isOk() at 85 °C
                changed.add(p);
            }
        }
    }
    UISession s = session;
    if (s == null) {
        return;
    }
    if (s.getState() == UISession.State.CLOSED) {
        close();                                          // the user is gone: stop the simulation
        return;
    }
    s.access(new Runnable() {                             // now on the session thread: listeners
may touch UI
        @Override
        public void run() {
            for (Part p : changed) {
                fire(p, false, false);
            }
        }
    });
}
```

`step()` runs every second on `SIM`, a daemon `ScheduledExecutorService`; the listeners it fires recolour the sensor meshes and the Parts tree refreshes its properties. No UI code knows there is a background thread. Thread-safe entry points that need no hop: `NotificationDisplayer.getDefault(session).notify(...)`, `ProgressHandle` (`createHandle(..., session)`), and `DashboardTopComponent.settingsChanged()`'s `d.getUI().access(...)` for reaching other sessions' windows.

12.4 Who is logged in

`UISession.current().getPrincipal()` gives name, display name and `hasRole(...)`. The Workbench status bar shows it; status-bar cells are registry entries on static factory methods:

```
@Registration(path = "StatusBar", name = "wb-user", position = 100, type = Component.class)
public static Component user() {
    final UISession session = UISession.current();
    final Label l = new Label(Bundle.LBL_User(session.getPrincipal().getDisplayName()));
    session.subscribe(UISession.LocaleChangeEvent.class, new Consumer<UISession.LocaleChangeEvent>()
{
        @Override public void accept(UISession.LocaleChangeEvent e) {
l.setText(Bundle.LBL_User(session.getPrincipal().getDisplayName())); }
    });
    return l;
}
```



The status bar of the demo: the status text on the left; on the right the signed-in user, the clock, the language switch (`Deutsch` / `English`) and `Sign out` — four `StatusBar` registry cells like the one above (`webbeans-demo/.../DemoStatusCells.java`) — then the platform's own notifications bell and connection indicator.

Administrators get the `SessionDirectory` service (`getSessions()`, `invalidate(id)`): the Workbench *Server Status* window lists every live session as nodes and its popup ends one. Accounts come from an `AuthenticationProvider` (the built-in properties file, seeded by `WorkbenchInstaller`; LDAP/OAuth providers implement the same interface — see `design/authentication.md`).

13. Internationalisation

- Strings: `@Messages({"KEY=Text with {0}"})` on the class → `Bundle.KEY(arg)`. Annotation attributes take `"#KEY"`. Add `Bundle_de.properties` next to the sources for translations (merged with the generated keys at build time).
- The session locale: `UISession.current().getLocale()`; `session.setLocale(new Locale("de"))` switches it live — menus, toolbars and dialogs relabel themselves, open windows get `localeChanged()`, and anything else listens for `UISession.LocaleChangeEvent` on the bus (the status cell above). `WorkbenchOptions` stores the choice; `SessionSetup` applies it at login.
- Formatting: use `DateFormat`/`NumberFormat` with the session locale (`OutputTopComponent.append`), never the JVM default.
- Help pages: a `page_de.md` next to `page.md` is served to German sessions (§15).
- Branding: `--branding workbench` makes the platform look up `Bundle_workbench.properties` before `Bundle.properties` everywhere, so an application can rename platform strings (*Help* → *Workbench Help*) without forking them.

14. Testing without a browser

`webbeans-test` provides `HeadlessUI`: a full session with the registry, the window manager and a fake transport, so a JUnit test opens windows, types, clicks, answers dialogs and inspects the deltas. The tutorial's tests:

```
@Test
public void helloGreetsByName() throws Exception {
    try (HeadlessUI h = HeadlessUI.start()) {
        h.open("HelloTC");
        h.type("hello-name", "Ada");                // the browser would send a "change" event
        h.click("hello-greet");
        assertEquals("Hello, Ada!", ((Label) h.byTestId("hello-greeting")).getText());
    }
}

@Test
public void addTaskActionAsksAndAddsANode() throws Exception {
    try (HeadlessUI h = HeadlessUI.start()) {
        TasksTopComponent tasks = (TasksTopComponent) h.open("TasksTC");
        final Action add = Actions.forID("Tasks", "org.webbeans.tutorial.AddTaskAction");
        h.run(new Runnable() {
            @Override public void run() { Actions.invoke(add, null, h.globalContext()); }
        });
        assertTrue(h.hasDialog());
        h.type("dialog-input", "Write tests");
        h.answerDialog("dialog-ok");
        Node[] kids = tasks.getExplorerManager().getRootContext().getChildren().getNodes();
        assertEquals(3, kids.length);
        assertEquals("Write tests", kids[2].getName());
    }
}

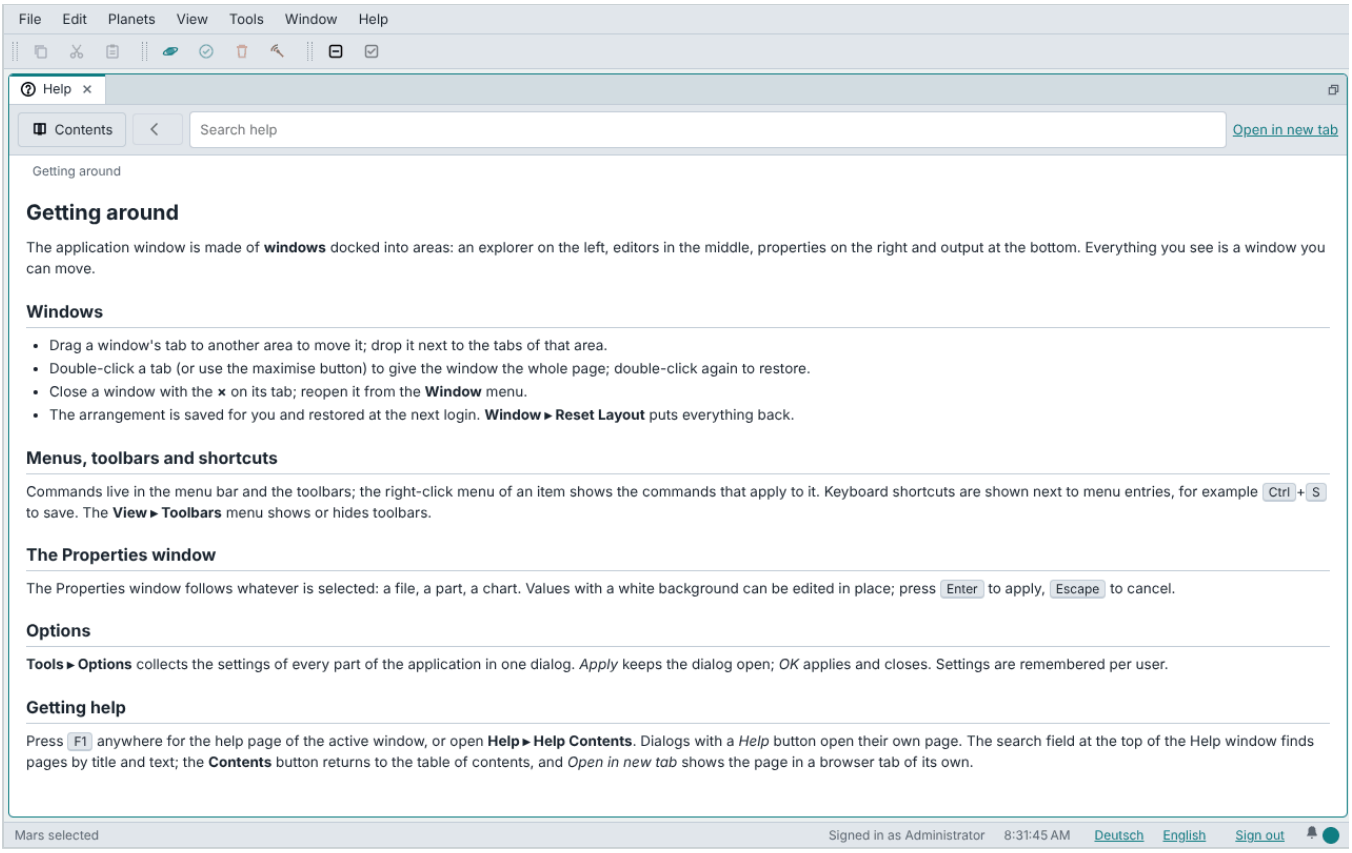
@Test
public void f1OpensTheTutorialHelpPage() throws Exception {
    try (HeadlessUI h = HeadlessUI.start()) {
        h.activate(h.open("HelloTC"));
        final Action f1 = Actions.forID("Help", "org.webbeans.help.ContextHelpAction");
        h.run(new Runnable() {
            @Override public void run() { Actions.invoke(f1, null, h.globalContext()); }
        });
        HelpTopComponent help = (HelpTopComponent) h.wm().findTopComponent("HelpTC");
        assertEquals("tutorial", help.getCurrentTopic().getId());
        assertTrue(help.getPageHtml().contains("<h1 id=\"tutorial-windows\">Tutorial windows</h1>"));
    }
}
```

The toolbox: `HeadlessUI.start()` (user `alice`, role `user`) or `HeadlessUI.builder().user("alice", "user", "admin").locale(Locale.GERMAN).dataDir(path).start()`; `open(tcId)`, `activate(tc)`, `openTab(id)` for a second browser tab; `run(Runnable)` to execute on the session thread; `type(testId, value)`, `click(testId)`, `inject(testId, eventName, data)` for any event; `hasDialog()`, `dialogButtons()`, `answerDialog("dialog-ok")`; `byTestId`, `all(Class)`; `messages()`, `lastDeltas()`, `callsOf(component, method)` to assert what went to the browser; `globalContext()`, `wm()`, `session()`, `ui()`, `userDir(moduleId)`. Everything the Workbench does is tested this way (`CoreTest`, `FilesTest`, `DashboardTest`, `ViewerTest`, `AdminTest`); the Playwright scripts under `workbench/e2e` and `webbeans-demo/e2e` cover what only a real browser can (rendering, drag and drop, WebGL).

Pattern — test through the same door the user uses. Open the window by id, trigger the action by id, answer the dialog by button — not by calling private methods. The test then documents the behaviour and survives refactorings.

15. Help pages

Users press **F1** in a window, click **Help** in a dialog, or open **Help ▸ Help Contents**; the Help window (its own area on the right) shows a page, with a table of contents, search, Back and *Open in new tab*.



The Help window showing the tutorial's "Getting around" page (maximised by double-clicking its tab). Contents, Back, the search field and Open in new tab are the window's own toolbar; the page is Markdown from the jar, styled with the application theme. A page is a Markdown file in your jar, registered with `@HelpPage` — the tutorial's `package-info.java`:

```
/** The tutorial of the user's guide — and its help page. */
@HelpPage(id = "tutorial", title = "#HLP_Tutorial", resource =
"org/webbeans/tutorial/help/tutorial.md", position = 500, keywords = "#KW_Tutorial")
@Messages({"HLP_Tutorial=Tutorial windows", "KW_Tutorial=hello, contact, tasks, guide"})
package org.webbeans.tutorial;

import org.webbeans.core.i18n.Messages;
import org.webbeans.help.HelpPage;
```

And the page, `src/main/resources/org/webbeans/tutorial/help/tutorial.md`:

```
# Tutorial windows

The windows of the tutorial follow the **User's Guide** (`docs/user-guide.md`):

| Window | Guide section |
|--------|---------------|
| **Hello** | §1 — a text field, a button and a label |
…
- **Tools ▸ Add Task…** (<kbd>Ctrl</kbd>+<kbd>Shift</kbd>+<kbd>T</kbd>) asks for a title and adds a
task.
```

15.1 Connecting pages to the UI

Where	How	Example
A window	`@TopComponent.Description(…, helpId = "tutorial")`, or `setHelpCtx(new HelpCtx("…"))` at run time	every Workbench window
A dialog	`descriptor.setHelpCtx(new HelpCtx("workbench"))` → a *Help* button; `F1` inside the dialog presses it	`AboutAction`
An options page	`OptionsPanelController.getHelpId()` → the Options dialog's Help button follows the category	`DashboardOptions`
Your own code	`new HelpCtx("files.editor").display()` (or `HelpCtx.DEFAULT_HELP` for the home page)	—

F1 shows the active window's page; a window without a `helpId` gets the **home page**, which is the first top-level topic by `position` (the Workbench core registers its overview at position 0).

15.2 Writing pages

- **Markdown** (`.md`): headings, paragraphs, lists, tables, code, links, images, block quotes; inline HTML such as `<kbd>` passes through. An `.html` fragment works too. Everything is sanitised before it reaches the browser.
- **A tree of pages**: `parent = "files"` nests a page under another (`files.editor` under `files`); `position` orders siblings. Pages registered by *different* jars can nest under each other — the Workbench jars all hang their pages under the core's `workbench` page:

```
@HelpPage(id = "files", parent = "workbench", title = "#HLP_Files", resource =
"org/webbeans/workbench/files/help/files.md", position = 100, keywords = "#KW_Files")
@HelpPage(id = "files.editor", parent = "files", title = "#HLP_Editor", resource =
"org/webbeans/workbench/files/help/editor.md", position = 100)
```

- **Links**: link to another page's *file* relatively — `[The editor](editor.md)`, or across jars `[Files](../../files/help/files.md)` — and the platform turns it into a link to that page's id; `help:files.editor` works too, and `#heading` anchors (the heading's id is its slug: *Working with files* → `working-with-files`). External links open a new browser tab.
- **Images**: `![diagram](pictures/flow.png)` next to the page; served from the jar.
- **Translations**: `tutorial_de.md` next to `tutorial.md` is used for German sessions; titles and keywords come from the bundle (`#HLP_Tutorial`).
- **Roles**: `roles = "admin"` hides a page from everyone else — in the table of contents, in search and on the stand-alone page (the Workbench's *Server Status* page).
- **Search** matches the title and keywords first, then the page text, so give pages a few `keywords` the title does not contain.

15.3 Outside the window

`GET /wb/help/<id>` serves any page as a complete HTML document with the table of contents — for a bookmark, a browser tab of its own or printing — to signed-in users, with the same role filtering and language. The *Open in new tab* link of the Help window points there. The platform ships one page of its own, *Getting around* (`help:platform`), about windows, menus, shortcuts and options; link to it from your overview.

Pattern — write the help where the feature lives. Each Workbench jar registers its own pages next to its code and tests; the core’s overview links to them by file. Adding a jar adds its help; removing it removes the pages and the links to them stop short rather than break the build.

16. Packaging an application

The Workbench shows the whole chain:

1. **Jars** — one each (`workbench-core`, `files`, `dashboard`, `viewer3d`, `admin`), depending on the platform jars and on `workbench-core` through Maven.
2. **Branding resources** — `workbench-app/src/main/resources/org/webbeans/actions/Bundle_workbench.properties` (and `_de`) override platform strings.
3. **The application jar** — `tools/fatjar.py` (or the `maven-shade-plugin` in `workbench-app/pom.xml`) merges the platform, the Workbench jars and the branding resources into one `workbench.jar` (`META-INF/services` files concatenated).
4. **Start script** — `workbench.sh`: `java -jar workbench.jar --branding workbench --title Workbench --datadir data --port 8080`. Other launcher options: `--host`, `--context /path`, `--locale`, `--session-timeout`, `--admin-password`, `--https-port 8443 --keystore server.p12 --keystore-password …` (TLS in the server; `--https-redirect` for plain requests), or `--secure-cookies`, `--trusted-proxy` behind a reverse proxy. Adding or upgrading application code means replacing jars on the class path and restarting.
5. **Data** — `<datadir>/system` (accounts, server state), `<datadir>/users/<name>/<package>` (preferences, layouts, the files jar’s working directory).

To start your own application: copy `workbench/` (Developer Guide §1.2) or `examples/hello`, rename the packages, delete the projects you do not need, keep `workbench-core` as the place for your shared API and installer.

17. The patterns, on one page

Pattern	Where you saw it
Build the UI in the constructor; subscribe in `componentOpened`, unsubscribe in `componentClosed`	§1, §6.1, `OutputTopComponent`
Enable the button from the validator's verdict (`e.isValid()`)	§2.1
Compose panels, place them with one `Layout.Border`	§3
UI timers for periodic UI work; `ui.access` to come back from a thread	§4, `DashboardTopComponent`, `PartsModel`
Dialogs are futures; `canClose` asks and returns `false`	§5, `EditorTopComponent`
Lookup in, actions out: windows publish objects, actions declare what they need	§6.4, §8, `FileActions.SaveAction`
Model → nodes → views; views never talk to views	§7, `PartsModel` + viewer + tree
Register actions in folders; nodes read the folder	§8, `TaskNode`, `FileNode`, `Scene3D/Actions`
Apply settings where they are used, re-apply at login	§9, `DashboardOptions`, `SessionSetup`
Bind collections with `Children.Keys` / `ChildFactory` / `SceneBinder`	§7.2, §11
Share across windows and tabs on the session bus	§12.2, `LogMessage`
Test through the user's door with `HeadlessUI`	§14
Write the help where the feature lives: `@HelpPage` + `helpId` per jar	§15, `workbench/*/package-info.java`
Declare validation on the component, aggregate with `Form`: min/max, required and formats are platform rules with localised messages; `form.enable(save)` and `form.validate()` replace hand-written enablement	§2.4, `GalleryTopComponent`
Let enums describe themselves (`Labeled`, a bundle line, `EnumLabels.register`) and every list, cell and editor shows the label and icon	§2.6, `Planet.Type`
Gestures arrive as events; the component applies what it owns and reports what you own	§6b, `BoardsTopComponent`

Where next: the Developer Guide for services, threading and the protocol in depth; the Component Guide when you need a property name; `design/*.md` for the why.